Computer Science
25COC251
F225694

Drug–Target Interaction Prediction Using Deep Neural Networks

by

Daniel R. Smith

Supervisor: Mohamad Saada

Department of Computer Science
Loughborough University

January 2026

Contents

Chapter 1

Introduction

1.1 Project Brief

1.1.1 Motivation

The pharmaceutical industry faces high costs and long development cycles in drug discovery, partly due to the challenge of identifying whether a novel compound can bind to a specific protein target. Machine learning (ML) provides a promising pathway for automating and accelerating this process by learning patterns from large bioactivity datasets.

1.1.2 Objective

The aim is to develop a predictive machine learning system that will determine the likelihood of binding between a new drug-like molecule and a protein target in the early stages of the process. Hopefully, reducing the long development cycles the pharmaceutical industry faces in drug discovery by reducing the sample size early on.

1.1.3 Computational Approaches in DTI

Drug-Target-Interaction was primarily tested during wet lab experiments, but the first *in silico*, computational approach, took place in 1982 with molecular docking of 3D structures (Ru et al. 2025). A later advancement to this was the breakthrough of using machine learning models, which have become increasingly more popular since the early 2000's, being able to autonomously recognise patterns and relationships from data. Ever since its introduction, there have been many attempts to use machine learning, but real progress was made in 2018 (Öztürk, Özgür & Ozkirimli 2018) when advanced deep learning models were created and tested, able to effectively learn representations from drug and protein sequence data.

1.1.4 Process

Data Acquisition

To extract and process relevant bioactivity data from BindingDB, a public database containing thousands of chemical compounds and protein target binding data.

Feature Engineering

Converting Chemical Compounds to molecular footprints using RDKit and encoding protein targets to numerical vectors using ProtBERT. These processes differ because the biological information is encoded in completely different forms, thus requiring domain-specific feature engineering methods.

Model Development, Training & Evaluation

The Neural Network will be developed and trained on the dataset to learn the underlying relationships that cause binders and non-binders. The model will be evaluated on validation metrics like AUC-ROC, PR-AUC, F1, Precision, Recall, and Accuracy. Based on these techniques, the Neural Network will be iteratively refined to reduce the error rate and improve accuracy.

Develop User Interface

The website for this project will be developed using the React framework due to its modular architecture, which provides a rapid development process and quick modifications with live rendering (Aggarwal 2018). The user interface will be designed to be straightforward, easy to navigate, and only contain necessary information relevant to the workflow. I want to display the result once it is defined, history of previous comparisons made, and their results, and to allow the user to compare multiple chemical compounds against one protein to mimic wet lab work.

Tech Stack

Component	Technology / Library
Dataset	BindingDB
Programming Language	Python, JavaScript
Data Processing	pandas, NumPy, scikit-learn
Drug Molecule Feature Engineering	RDKit (molecular fingerprints, SMILES parsing, descriptor extraction)
Protein Feature Engineering	ProtBERT (protein embeddings using transformer-based model)
Neural Network Framework	PyTorch (model definition, training, and evaluation)
Model Evaluation	Scikit-learn
User Interface	React (frontend framework for model interaction and visualization)
Backend API	FastAPI (model serving and communication with frontend)
Version Control	Git / GitHub

Table 1.1: Technologies and tools used in the project.

1.2 Feature List

This is where I will show you all the features I wish to include within my project and what part they play in the project.

1.2.1 Required

Compare a single drug molecule to a single protein target

This will be the first and most basic functionality of the product as it will allow the user to search the ChEMBL database for a single drug molecule to test against a single protein target.

Compare multiple drug molecules to a single protein target

I will achieve this by allowing the user to multi-select drug molecules or selecting a drug based on its standard type and relation to filter it into groups. This will give the user a closer feel for what wet-lab work could achieve.

Displaying previous tests

Having a section which displays what the user had previously tested is important as it would reserve computational resources for new combinations to compare, preventing the user from re-using the neural network to repeat a binding.

1.2.2 Nice-to-Have

Produce indirect links into how they bind

Once the neural network has run and the result has been displayed to the user, I would try and also include a brief explanation into why the network predicted what it predicted.

Displaying molecule and Protein information

Retrieving this relevant information from the BindingDB database so the user can see more information about the compound-protein pair they have selected.

1.3 Scope & Limitations

1.3.1 Project Scope

To ensure a complete project and to counter issues that may spring up, each milestone reached will provide a submittable piece of work that can be a good base to work off for the next stage:

Predicting drug-target interactions

Developing a NumPy model capable of making predictions of drug-target interactions by classifying them into binding/non-binding.

Using neural networks for prediction

Implementing a training a neural network architecture to learn relationships between molecule and protein features, producing consistent and reliable predictions.

Including both chemical compound and protein sequence features

Extracting and integrating molecular descriptors or fingerprints for chemical compounds and sequence-based or embedding-based features for protein targets.

Implementing a web-based UI for result visualization and comparison

Creating a simple, React-based web application to visualize prediction results, allow compound-target comparisons, and provide basic interpretability.

1.3.2 Project Limitations

Here is what is retracting from the project and preventing it from being as good as it could possibly be.

Data limitations

Due to the nature of the project, I am restricted to only using publicly available datasets (e.g. BindingDB, ChEMBL) that may have missing or biased data from their producers. Also, the data set might not be complete, missing vital information which could improve the models' success rate.

Model limitations

Neural networks might not capture rare interactions or out-of-distribution compounds, and they don't take the structure of each molecule into account. A Graph neural network (GNN) could be used, but that gives me greater limitations computationally and data wise.

Computational constraints

I am only limited to what I have exposure to, and the model will be trained on my personal laptop, so my computational resources may not be representational of what big pharma companies have access to, producing a considerable gap into what the industry could be exposed to.

UI limitations

This is only a simple prototype, so I'm not optimising it for a production scale but will make it suitable for a small team to use.

1.3.3 Assumptions

For this project I am assuming the only people using it will have sufficient prior knowledge within chemistry to use this product. I am assuming the data within BindingDB is accurate and up to date.

1.4 Workflow

In this section i would like to display a visual representation on how I will take this project from start to finish.

1.4.1 System Overview

The system predicts potential interactions between chemical compounds and protein targets using a trained neural network. It processes input data, extracts numerical features, performs predictions, and displays the results through a web interface for comparison and interpretation

Component	**Description**
Data Collection & Preprocessing	Collecting compound and protein data from public databases such as BindingDB, and filtering it based on the required parameters.
Feature Engineering	Converting molecular structures and protein sequences into numerical feature representations such as fingerprints & embeddings
Model training and Evaluation	Training a neural network on the processed dataset to pickup on relationships and evaluating the performance with metrics such as AUC-ROC and F1 score.
User interface	Displays the result, history and allows the user to customize their comparisons. Built with react for responsiveness and ease of use.

Table 1.2: System components and their descriptions.

1.4.2 Workflow Diagram

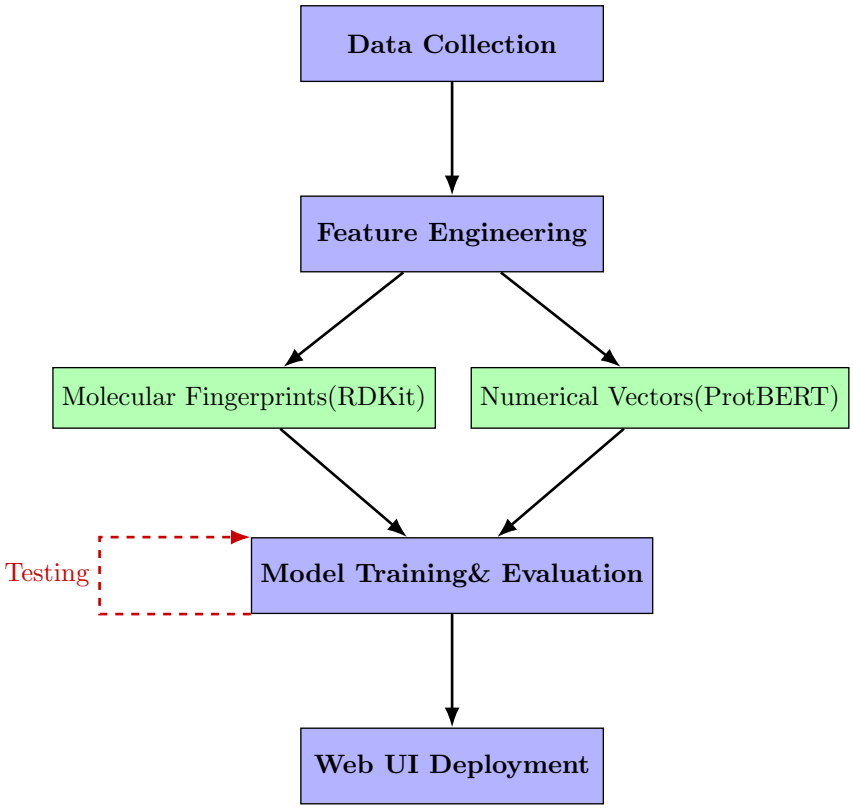

Figure 1.1: Top-down project workflow illustrating parallel feature engineering and iterative model refinement.

1.4.3 Project Timeline

Use of a Gantt chart to keep on top of things.

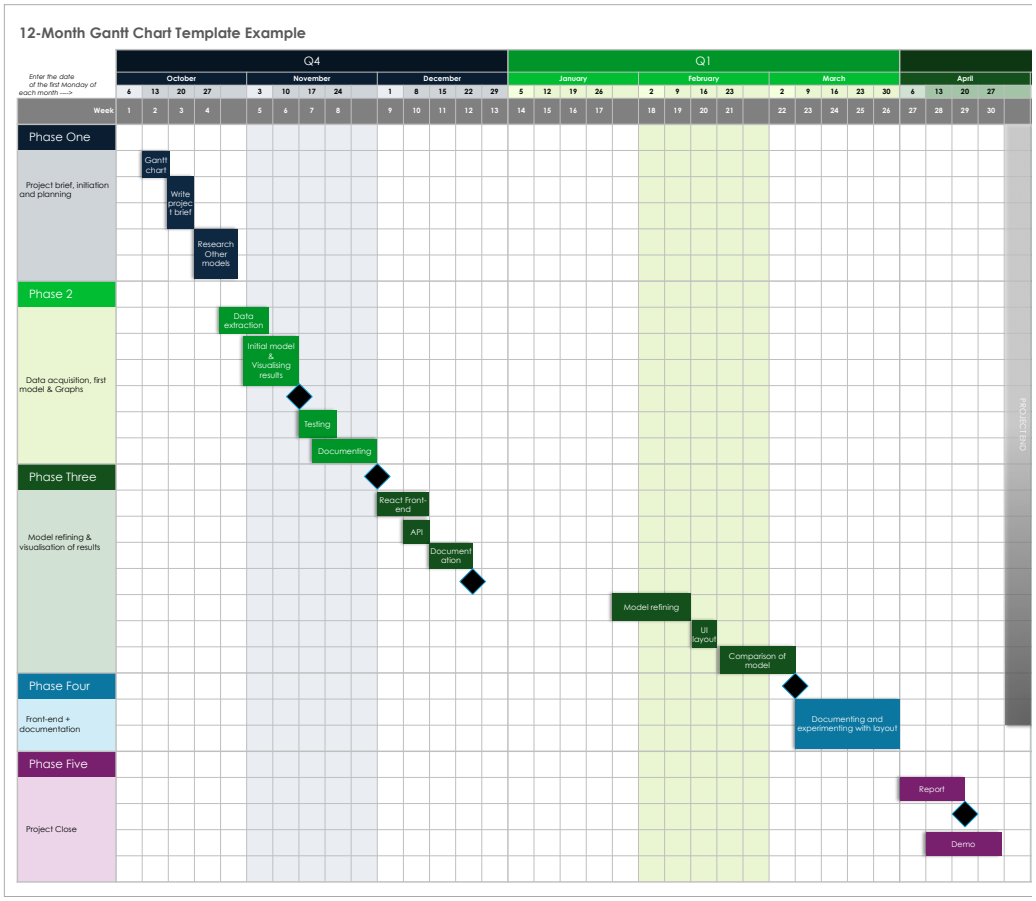

Figure 1.2: Project Gantt chart showing major milestones and development phases.

Chapter 2

Background Research

This chapter reviews the current state of research and available tools in the field of drug-target interaction prediction. The review is divided into **(i)** foundational concepts, **(ii)** current machine-learning approaches, **(iii)** existing prediction tools, and a **(iv)** gap analysis highlighting the need for the proposed project.

2.1 Initial concept

Drug discovery is the process through which potential new therapeutic entities are identified, using a combination of computational, experimental, translational, and clinical models (Zhou & Zhong 2017). Drug-Target Interaction (DTI) prediction is a critical aspect of drug discovery, as it helps in identifying potential drug candidates that can interact with target proteins to exert their desired therapeutic effects (Yang et al. 2024). A drug is a chemical compound which causes physiological changes in the body when consumed, injected, or absorbed. A target, also known as a biological target, is a structure located in an organism that is recognised or bound by other substances such as ligands or drugs and can be acted upon by a drug or other targeted molecules (Overington et al. 2006). The pharmaceutical industry faces mounting financial pressures and long development cycles in drug discovery, which creates a huge challenge when discovering new drugs, especially when they are needed in short time spans, as seen and experienced with COVID-19. One way to combat these challenges is to use machine learning to potentially automate the initial screening process, reducing both equipment and labour costs. This approach is invaluable in the initial stages of drug discovery when the sample size is extremely large and can be accurately whittled down by an algorithm trained to predict interactions based on known patterns.

2.2 Foundations of DTI Machine Learning

At the beginning of DTI using machine learning, methods included similarity-based approaches. This is where drugs with similar structure are assumed to interact with similar proteins. Feature-based machine-learning models are much more refined and rely on more scientific knowledge compared to just their structure, such as fingerprints or protein sequence features. These come in two different ways, **Regression**: where the model predicts continuous binding affinity and **Binary classification**: where affinity values are put into thresholds, active or inactive.

2.3 Deep learning models in DTI

Deep Learning models have become a strong way to determine interactions, due to its ability to learn high-dimensional representations from raw data. There are many approaches for deep learning architectures, for example, the use of fully connected neural networks (FCNN), (Talukder et al. 2025, Zhan et al. 2025) which is a layered network that connects every neuron in one layer to every neuron in the next layer and evaluated using Accuracy, Precision, Recall, AUC-ROC, and AUC-PR.

2.3.1 Sequence-Based Models

A Sequence-based model encodes raw data, Drug SMILES, and Protein amino acid sequences, directly. Recurrent Neural Networks (RNN) and Convolutional Neural Network (CNN) are used to learn patterns in local chemical environments or amino-acid motifs. Models like DeepDTA (Öztürk, Özgür & Ozkirimli 2018) display impressive predictive performance using sequence-only architectures.

2.3.2 Graph-Based models

Graph Neural Networks (GNNs) take as input the structure of molecules as graphs of atoms, learning structural information which, to sequence-based models, is challenging to learn. Yang et al. (2024) utilised this as part of their algorithm using Message-Passing Neural Network (MPNN) to encode drug sequences as a part of a greater system. An MPNN can extract more detailed relational features, for improved modelling of chemical structure.

2.3.3 Hybrid Networks

A Hybrid network is the combination of multiple neural network architectures to capture complimentary information to improve the learning rate of the model. The hybrid model

in (Yang et al. 2024) uses an MPNN to encode drug sequences and a Convolutional Neural Network (CNN) to encode protein sequences. To generate the other prediction, it uses a High Order Graph attention Convolutional Network (HOAGCN), which makes the prediction based on the structure of the drug molecule and target protein. All these combined make for a much more meaningful prediction producing accurate results.

2.4 Binary Classification in DTI

There's lots of research and studies that have been done using binary classification for the interaction task, this is where the model outputs whether it will (1) or it won't (0). This has become increasingly more popular as it simplifies the learning objective, allowing the model to spend more computational power on classification metrics.

2.4.1 Research Papers Using Binary Classification

A strong model which uses this type of classification is DeepDTA (Öztürk, Özgür & Ozkirimli 2018), which applies a threshold to continuous affinity values (IC50 or Ki) and classifies the interactions as active or inactive. Although the study also explored regression, it showed that the binary classification improves model stability and reduces noise introduced by inconsistent conditions. Another strong case is produced in Talukder et al. (2025), where explicit affinity binarization combined with MACCS fingerprints and protein sequence embeddings are used to train a neural network that outputs a binary interaction score. This is supported by Zhan et al. (2025), who follows the same approach, emphasising the robustness in binary prediction when affinity values are inconsistent.

2.4.2 Affinity Thresholds

The affinity thresholds used are a critical design choice because it determines the data the model ingests and the classifications made. Having an off-centered threshold can skew the class distribution even if the underlying dataset is balanced. DeepDTA (Öztürk, Özgür & Ozkirimli 2018) established threshold conventions that have since become standard in DTI benchmarking: a pKd ≥ 7 cutoff for binders on the Davis dataset, and a KIBA score ≥ 12.1 cutoff on the KIBA dataset. These thresholds have been widely adopted in subsequent DTI work (GraphDTA (Nguyen et al. 2021), DeepMHADTA (Deng et al. 2022), MINDG (Yang et al. 2024)), making them the standard for cross-study comparability.

2.4.3 Negative Sampling

Some of the recent work that uses binary classification also highlight the importance of negative sampling during training. Since many DTI datasets tend to be weighted towards positive pairs, the data becomes skewed. Several recent models, including DeepPurpose (Huang et al. 2020) and other BindingDB approaches, address skewed data using negative sampling to generate synthetic pairs to avoid having an unbalanced training dataset.

2.4.4 Evaluation metrics

Commonly seen among these studies in binary classification,(Huang et al. 2020, Talukder et al. 2025, Öztürk, Özgür & Ozkirimli 2018) is the reliance on **(i)** Accuracy, how many observations were correctly classified **(ii)** Precision, How many positive predictions were actually positive **(iii)** Recall, how many positive cases did the model correctly predict **(iv)** AUC for either **(i)** PR, A chart that visualises the Precision and Recall in a single graph or **(ii)** ROC, is a chart that visualises the trade-off between the true positive rate (TPR) and the false positive rate (FPR).

2.4.5 Binary Classification Summary

Binary classification has become the basic method for contemporary DTI research because it can complete the tasks at machine-learning benchmarks but in a more simple way. This is why I have decided to pursue a model that works with binary classification and use classification-based evaluation metrics.

2.5 Dataset & Feature Engineering

Recent research papers consistently take as input drug SMILES and proteins as Amino Acid Sequences from BindingDB's dataset (Vefghi et al. 2025), reinforcing the fact that this is the most appropriate source of data in DTI. A few studies differ mainly in how they encode these inputs. For example, the paper by Talukder (Talukder et al. 2025) represented the drugs as MACCS fingerprints generated by RDKit, these are fast, interpretable and straightforward but are not very expressive (Xie et al. 2020), making them easy to compute which can benefit me with limited computational power. Protein features are similarly derived from amino acid sequences through embedding frameworks such as ProtBERT.

BindingDB was selected over alternatives such as ChEMBL due to its scale, heterogeneity of affinity measurements, and frequent use in recent DTI literature (Talukder et al. 2025,

Yang et al. 2024). ChEMBL offers comparable scale but is oriented more toward general bioactivity rather than binding affinity.

2.6 Common limitations

What seemed to be a common limitation was the lack of data on successful binds creating an imbalanced dataset. However, some of these papers tried to combat this issue by taking data from two datasets (Yang et al. 2024) or manipulating the data through affinity harmonization & binarization (Talukder et al. 2025) and a Polyloss framework (Leng et al. 2022), which is an addition to standard loss functions that allows us to precisely tune how the model handles our severe class imbalance.

2.7 Current products

2.7.1 SwissTargetPrediction

The website SwissTargetPrediction (*SwissTargetPrediction* n.d.), allows the user to predict using a ligand-based approach built to combine different similarity measures, providing more accurate predictions. The method, as described in the 2013 paper (Gfeller et al. 2013), was trained on a set of 224,412 molecules known to interact with 1700 human proteins from the ChEMBL database.

For many bioactive molecules their primary target is unknown, and even when it is, most have more than one target (Off-targets) that are not well characterised. Making predictions for these off-targets is important for understanding side effects and drug repositioning – finding new uses for old drugs.

The method's main strength is its hybrid approach to similarity, combining multiple variables to discover unknown targets of similar structure. It uses and combines **i** 2D Chemical Similarity: This uses FP2 chemical fingerprints to find structurally similar molecules, which is effective for identifying analogues (structurally similar molecules) within the same chemical series. **(ii)** 3D Shape Similarity: This uses Electroshape, which is a 3D method that compares molecules based on their shape, atomic partial charges, and lipophilicity. This is powerful for finding active molecules with different chemical structures.

These two scores are weighted and combined using a multiple logistic regression model. The authors demonstrated that while 2D fingerprints alone perform well on biased datasets, the combined 2D/3D approach is significantly more accurate in the more realistic scenario of predicting targets for novel molecules that do not belong to a known chemical series.

The interface is plain and a bit scattered, but it's easy to use; you select a species (Homo

sapiens, Mus musculus, Rattus norvegicus), and then choose a drug molecule through a given example, SMILES representation, or drawing the molecule. Which is a creative and more advanced way to really test new drug compounds and structures.

2.7.2 DINIES

The web tool DINIES (Yamanishi et al. 2014, *DINIES: Drug-target Interaction Network Inference Engine based on Supervised Analysis* 2014), predicts potential interactions based on drug data and omics-scale protein data, allowing the user to use whatever data as input as long as it's represented accordingly. First published in 2014 it uses machine learning and predicts using the *guilt-by-association* principle, similar drugs are likely to interact with similar proteins.

The methodology is based on using Kernels to convert chemical structure, side effects, and Protein sequence profiles to separate matrices. The server combines the matrices and in its simplest mode will just average them together to create a single combined drug similarity matrix.

However, this foundational machine learning method has become very limited compared to other products as it over relies on features, and it only learns simple mapping, not complex mapping, or non-linear relationships from raw data. Furthermore, the UI is not designed for simple queries as it requires the user to format and upload their own data matrices, creating a high barrier to entry for most researchers.

2.8 Gap analysis

After a detailed review of Drug-Target interaction products and papers, from foundational similarity-based methods to modern deep learning models, three gaps in the current landscape have appeared: (i) **SOTA vs Usability Gap**, gap between modern, cutting-edge research models and tools accessible to the public (ii) **Novelty problem**, similarity-based models will struggle when given novel drugs or targets which have no known analogues (iii) **The Data Imbalance gap**, older datasets were imbalanced and some using conventional loss functions to counteract, but they can turn out biased, resulting in poor performance.

My proposed predictive ML pipeline will be designed to directly address these three gaps, **(i)** It will deliver SOTA accuracy by using modern neural network architectures to learn representations reducing the novelty issue **(ii)** using weighted loss functions to force the model to focus on hard-to-find positive interactions to address the data imbalance challenge **(iii)** diminishing the usability gap by connecting powerful backend to a simple UI, displaying results and information in an accessible way, a feature missing from most complex models.

Chapter 3

Data Pipeline

3.1 Introduction

This chapter provides a detailed overview of how the data was sourced, processed, and prepped for two inputs, proteins and drugs, turning them from raw data to computationally readable for the model.

3.2 Data Collection and Curation

Data is an extremely important part of this project, and it's important to gather a sufficient amount of scientifically accurate data so the model can be the most accurate as possible. This section will define how the data was sourced and manipulated to reach its final representation.

3.2.1 Primary Data Source

I extracted the initial bulk of my data from BindingDB (University of California San Diego 2025), which is a public database holding experimentally measured binding affinities between small molecules and proteins (Liu et al. 2025). BindingDB serves as a critical resource to a diverse range of applications, including development of artificial intelligence models. The database has grown significantly, holding information on 2.9 million binding measurements for 1.3 million compounds. Data can be extracted through its web-services or downloaded directly. Due to the nature of the project, the initial phase begins with a local download of two raw data files to facilitate data cleaning and binarization:

- `BindingDB_all.tsv` file containing millions of interaction records

- `Sequences.fasta` file containing the amino acid sequences for all protein targets in the database.

3.2.2 Data Merging and Cleaning

The raw data from BindingDB didn't contain protein sequences, making it unsuitable for feature engineering, so the first step after downloading all raw data was to unify the interaction and protein sequence data. Using the protein name as a common key, the `.tsv` and `.fasta` file were merged, creating a unified dataset where each row contained **Drug SMILES**, **Target Sequence**, **Binding Affinity Value**, and the **Interaction classification**.

BindingDB reports binding affinity across several measurements: Ki, Kd, IC50, and EC50. But most rows contain only one or two of these values, as illustrated in Table 3.1. Ki and Kd are preferred as they measure direct binding affinity, whereas IC50 and EC50 are functional measures that depend on assay conditions, making them less directly comparable (He et al. 2017). To maximise data coverage, we adopted a priority-based fallback: Ki is used when available, otherwise Kd, then IC50, and finally EC50. The selected value is converted into a log-scaled representation described in Subsection 3.2.5, compressing the wide range of raw nanomolar values into a more stable scale, *pAffinity*, for model training. A full justification of affinity type selection is provided in Subsection 3.2.4.

Mixing heterogeneous affinity types like this is a known limitation of this approach, as the absolute values are not directly comparable across measurement types. But it is partially mitigated with a grey-area buffer used during binarisation, to only allow confidently strong binders and clear non-binders to receive labels.

During the merge, the data was also cleaned to remove duplicate interactions, any rows with NULL values, and where affinity ≤ 0. The interactions were filtered to only include *Homo sapiens* targets, because the goal is to produce therapeutic drugs for humans and cross-species affinity is not transferable. This keeps the distribution of pAffinity values more homogeneous and allows comparability with published benchmarks (Öztürk, Özgür & Ozkirimli 2018, Yang et al. 2024). The result was a unified table containing the necessary information to continue with feature engineering.

A snippet of the raw `.tsv` file is shown in Table 3.1.

Table 3.1: Snippet from `bindingdb_all.tsv` (Raw)

Drug SMILES	**Target Name**	Ki (nM)	Kd (nM)	IC50 (nM)	EC50 (nM)
`CC(=O)Oc1...`	Thymidine kinase	50.0	—	—	—
`C[N]1C=C...`	Thymidine kinase	—	—	12000.0	—
`CC(C)(C)OC...`	ABL1 kinase	—	75.0	—	—

The `.fasta` file was parsed into a key-value dictionary to map protein names to their sequences, as shown in Table 3.2.

Table 3.2: Example of Parsed `binddbtargetsequence.fasta` Data

Key (from FASTA Header)	Value (Sequence)
Thymidine kinase	`MPK...` (sequence truncated)
ABL1 kinase	`GKL...` (sequence truncated)
...	...

These two sources were merged on the target name. After merging, cleaning, and binarization, the final unified dataset was created and a snippet shown in Table 3.3.

Table 3.3: Snippet of Final Merged and Processed Dataset

`drug_smiles`	`target_sequence`	`target_name`	`p_affinity`	`interaction`
`CC(=O)Oc1...`	`MPK...`	Thymidine kinase	7.30	1
`C[N]1C=C...`	`MPK...`	Thymidine kinase	4.92	0
`CC(C)(C)OC...`	`GKL...`	ABL1 kinase	7.12	1

3.2.3 Drug Identifier Normalization

Drug identifiers provided by BindingDB corresponded to experimental compounds, usually lists of numbers that are lengthy and opaque to non-specialist users, instead of approved and recognised drug names. To improve the usability if the virtual screening interface and enable cross-referencing against external drug databases, the identifiers were normalised to be in simpler forms. The original identifier was retained to ensure traceability back to the BindingDB database.

```
def normalize_drug_name(self, name: str):
    if name is None or pd.isna(name):
        return None
```

```
    name = str(name).strip()
    if not name:
        return None
    parts = [p.strip() for p in name.split("::") if p.strip()]
    if not parts:
        return None
    parts = sorted(parts, key=len)
    for p in parts:
        if not re.search(r"[=#\[\]\(\)]", p):
            return p
    return parts[0]
```

Listing 3.1: drug name normalisation from BindingDB identifiers

3.2.4 Affinity Type Justification (Ki, Kd, IC50)

BindingDB already provides some experimentally measured binding affinity types, such as *Ki*, *Kd*, and *IC50*. Although these quantities are related, they do not supply us with meaning and are not suitable to begin our classification of the dataset.

Definitions and Mechanistic Differences

Ki (inhibition) measures the binding between an inhibitor drug to an enzyme, which strictly reduces the catalytic activity of the enzyme. **Kd** (dissociation) also measures the binding equilibrium between a drug and protein but in a more general, all-encompassing term to include all the dissociated components. Both Ki and Kd are equilibrium constants governed by the same thermodynamic principles (The Science Snail 2019). **IC50** is short for inhibitory concentration 50%, meaning the concentration of inhibitor required to reduce the biological activity of interest to half of the uninhibited value. The simplicity and lack of assumptions required to generate these data make them useful, but as it doesn't directly measure the binding equilibrium, it is a less accurate measurement than Ki or Kd. The relationship between IC50 and Ki is described by the Cheng-Prusoff equation, which shows that IC50 varies with substrate concentration even when Ki remains constant (Cheng & Prusoff 1973). **EC50** stands for effective concentration 50%, which refers to the concentration of drug at which 50% of its maximum effect is achieved. Making it more effective for experiments wanting the dosage to be quantified even when the enzyme activity doesn't reach the 50% threshold causing IC50 to be undefined. It reflects functional activity rather than direct binding affinity. EC50 measurements incorporate complex factors including receptor reserve, signal transduction efficiency, and cellular response mechanisms (The Science Snail 2019).

Justification for Ki Selection

To evaluate which affinity type is most appropriate for my model, we analysed the BindingDB dataset across four key criteria: mechanistic validity, data quality, measurement reliability, and data availability. **Mechanistic Validity:** Ki and Kd represent true equilibrium binding constants that are substrate-independent and thermodynamically well-defined. In contrast, IC50 and EC50 are functional assays whose values depend on experimental conditions and do not directly measure binding affinity. For a machine learning model intended to learn binding relationships, using equilibrium constants provides a more robust and interpretable basis (Figure 3.2A). **Data Quality:** After reviewing the graphs generated from the raw bindingDB file, Ki has the lowest outlier rate, which indicates more consistent experimental measurements compared to IC50 and Kd. Higher consistency is better for training the model as it's reducing noisy and inconsistent labels from the dataset (Figure 3.2B). **Data Availability:** While IC50 measurements are more abundant in BindingDB (47,043 entries), Ki still provides 16,796 high-quality measurements. This is plenty to develop and train machine learning models as modern deep learning models have shown it's more valuable to have high-quality data than a higher quantity (Northcutt et al. 2021). **Cross-Validation with Correlation Analysis:** To confirm the relationships between affinity types, a correlation heatmap was generated across all measurements in the raw BindingDB dataset (Figure 3.1). The high correlation between Ki and Kd ($r = 0.94$) confirms their mechanistic similarity as equilibrium constants. The moderate correlation between Ki and IC50 ($r = 0.81$) reflects IC50's dependence on experimental conditions, while the weaker correlation with EC50 ($r = 0.55$) highlights its distinct mechanistic basis as a measure of functional activity rather than binding affinity.

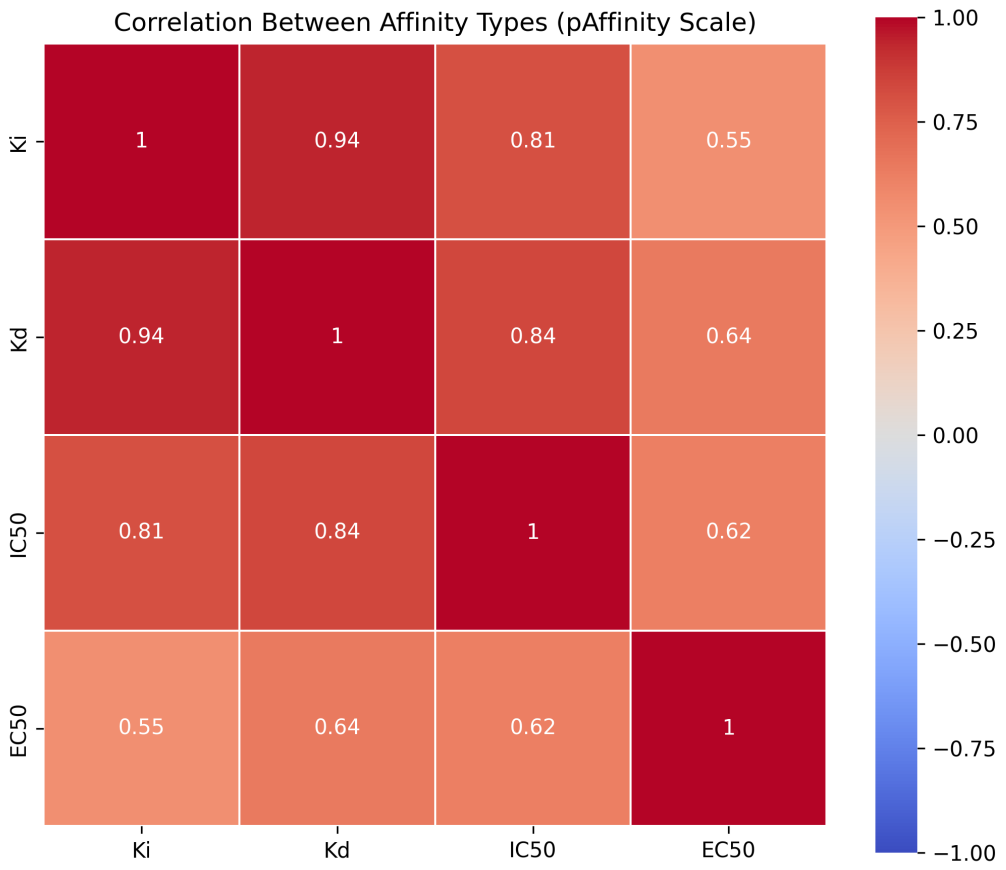

Figure 3.1: Heatmap showing the correlation between the different affinity values to show similarities in the values given

Based on this analysis, **Ki was selected as the primary affinity measurement** followed by Kd then IC50 as Kd will provide more accurate data than IC50, but it is not present enough to be used initially. Ki provides the perfect balance of mechanistic validity (true equilibrium constant), data quality (lowest measurement variability), and sufficient quantity (16,796 measurements) for reliable DTI classification.

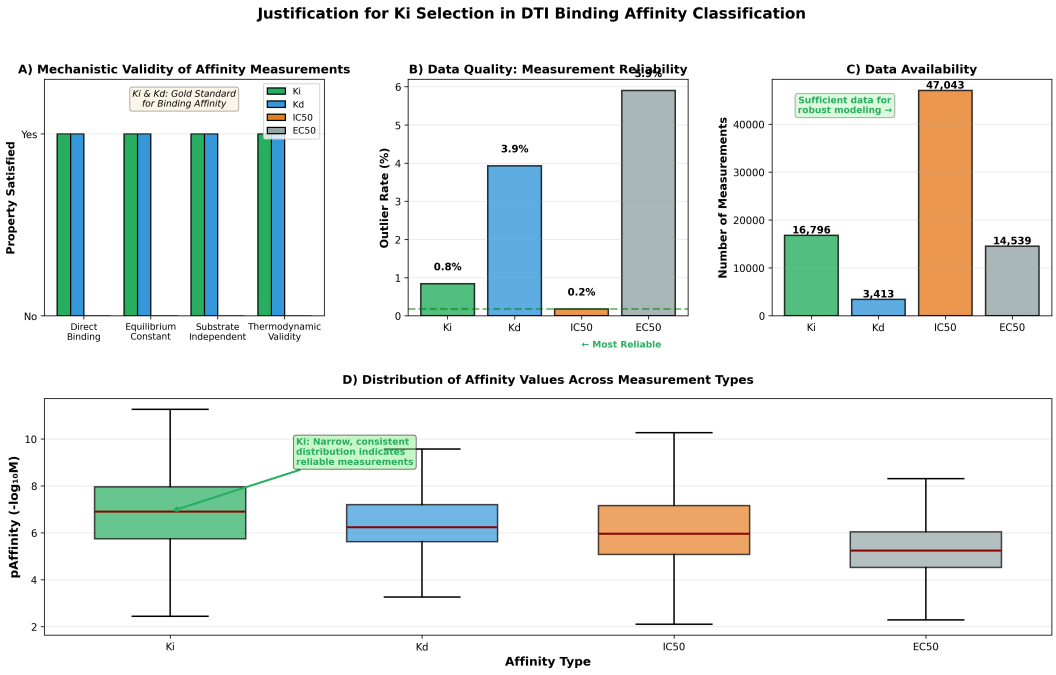

Figure 3.2: Comprehensive justification for Ki selection. (A) Mechanistic validity comparison showing Ki and Kd satisfy all criteria for true binding measurements. (B) Data quality assessment via outlier rates. (C) Data availability across affinity types. (D) Distribution of pAffinity values showing Ki's consistent measurement characteristics.

3.2.5 Data Binarization & Negative Sampling

It's crucial to classify the data accurately with meaningful representations and thresholds. From the BindingDB dataset we used the columns **Ki** & *Kd* and set thresholds on this value to determine the score binder or non-binder. Instead of using raw nanomolar (nm) values, the binding affinities were transformed into their log-scaled form using equation 3.1:

$$\text{pAffinity} = -\log_{10}(\text{Affinity}(\text{M})) \tag{3.1}$$

Where the raw nanomolar (nM) value is first converted to molar (M) units using equation 3.2:

$$\text{Affinity}(\text{M}) = \text{Affinity}(\text{nM}) \times 10^{-9} \tag{3.2}$$

This representation is widely used in the industry and literature (Thafar et al. 2022, D'Souza et al. 2023, He et al. 2017, Öztürk, Özgür & Ozkirimli 2018) due to its biological interpretability and numerical stability.

A common challenge with data in DTI is the insufficient number of true negative data as databases are biased towards successful binding experiments. A basic solution to this is random negative sampling, which pairs drugs and proteins at random and assuming they do not interact. However, this can introduce a high rate of false negatives and degrade model performance, which is highlighted as an issue of *low-confidence negative samples* in (Bai et al. 2021). Methods like fuzzy-rough approximation (Islam et al. 2021), select negatives by degree rather than at random, providing a better alternative for this type of sampling.

To counter this issue, we defined two binding thresholds and used negative sampling with care by following the *safe-margin* practice described in (Bai et al. 2021). Therefore, we classify pairs with pAffinity value > 7 as a binding interaction (label $= 1$), and pairs with pAffinity value < 5.3 as non-binding (label $= 0$).

3.2.6 Implementation of Affinity Transformation and Label Assignment

```
df_chunk = df_chunk[df_chunk['affinity_value_nm'] > 0].copy()
# Convert to pAffinity (-log10)
df_chunk['p_affinity'] = -np.log10(df_chunk['affinity_value_nm'] * 1e-9)
# Create Binary Labels (-1 = invalid, 0 = non-binder, 1 = binder)
df_chunk['interaction'] = -1
df_chunk.loc[df_chunk['p_affinity'] > BINDER_THRESHOLD_P, 'interaction'] = 1
df_chunk.loc[df_chunk['p_affinity'] < NONBINDER_THRESHOLD_P, 'interaction'] = 0
# Keep only valid 0 or 1
df_chunk = df_chunk[df_chunk['interaction'].isin([0, 1])]
```

Listing 3.2: Affinity transformation and binary label assignment

This approach generates an area between the thresholds, which represents an ambiguous *grey area*, a zone where the interactions are too week to be a binder but too strong to be a clear non-binder. To increase the performance of the model, we only want to train it on the clearest signals, so all pairs inside the *grey area* are removed from the loaded dataset shown in Table 3.4, and a distribution graph visualising the grey area is shown below in Figure 3.3.

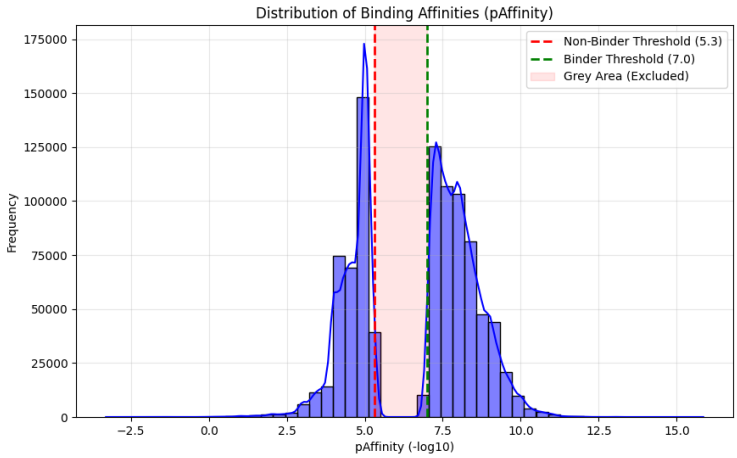

Figure 3.3: Distribution of pAffinity values in the filtered dataset. The red shaded region represents the *Grey Area* ($5.3 \leq$ pAffinity ≤ 7.0), where data points were excluded to ensure high-confidence labels.

Table 3.4: BindingDB Dataset Summary Before Thresholding

Metric	**Count**	**Percentage**
Total interaction pairs	1,416,025	–
Binders (pAffinity > 7.0)	552,618	39.03%
Non-binders (pAffinity < 5.3)	356,025	25.14%
Grey area ($5.3 \leq$pAffinity ≤ 7.0)	507,382	35.83%

After removing the 507,382 pairs falling within the grey area, the remaining interactions were assigned binary labels, resulting in the class distribution shown in the Table 3.5.

Table 3.5: Final Class Distribution After Thresholding

Class Label	**Interaction Type**	**Threshold Criteria**	**Count**	**Percentage**
1	Binder (Positive)	pAffinity > 7.0	552,618	60.82%
0	Non-Binder (Negative)	pAffinity < 5.3	356,025	39.18%
Total			**908,643**	**100%**

A subsampled copy with 18,520 interactions was used for the iterative experiments. This allowed for more appropriate training times with NumPy backpropogation. This was split

train/val/text as 12,038/3,704/2,778, and the distribution was mentioned in Table 6.1 in the evaluation section.

3.3 Feature Engineering

This section outlines the Feature engineering process to turn raw chemical and biological data into machine-readable formats. Drug SMILES and Target Sequences must be encoded into fixed-size numerical vectors as neural networks cannot process the raw text representations.

3.3.1 Drug Representation

Drug SMILES being represented as variable length strings are unsuitable for the model to ingest. To address this issue, drug SMILES were encoded into MACCS keys, a 167-bit binary fingerprint, where each bit represents the presence of a different chemical substructure.

3.3.2 RDKit

The open-source Cheminformatics toolkit RDKit performed *Descriptor Generation* by parsing SMILES strings into *Mol objects*, translating complex chemical structures into a fixed length vector (MACCS Keys) that the model can use to learn patterns for each drug & protein pair (RDKit Contributors 2025). As a vector notation:

$$\text{MACCS} = [0, 0, 1, 0, 0, 0, 0, 0,\ 1, 0, 0, 0, 0, 0, 0, 0,\ 1, 0, 1, \ldots] \in \{0, 1\}^{167}.$$

3.3.3 Protein Representation

The Proteins are represented as amino acid sequences of varying lengths, often exceeding thousands of characters. To normalise the sequences, so they're suitable for the neural network, the variable-length sequences were converted into fixed-size continuous embeddings.

3.3.4 ProtBERT

To ensure the embeddings were as accurate as possible, ProtBERT was used as it's a pre-trained transformer model based off of the BERT architecture (Devlin et al. 2019). ProtBERT understands the contextual and biophysical properties of the amino-acid sequences as it has been trained on a huge database of protein sequences UniRef100 (Consortium 2025) containing 217 million sequences (Elnaggar et al. 2020). By extracting the hidden states from the model through mean pooling, each protein sequence is condensed into a semantic vector embedding that fits the fixed-size, *1024 floating-point values*, input and preserves the biological context, such as 3D structure or chemical polarity (Elnaggar et al. 2020).

Protein embedding (example, size 1024):

$$\mathbf{p}_{\text{ex}} \in \mathbb{R}^{1024} = [0.320856,\ -0.685886,\ -0.652373, \ldots,\ -0.344068]$$

3.4 Data Preparation

Following feature engineering, the processed data was split into 3: *Training, Validation* and *Testing*. This is an important step to prevent overfitting of the model on the dataset, keeping this randomised is key to the validity of the model.

3.4.1 Train-Validation-Test Split Methodology

To ensure reproducibility, the dataset was split similarly to Arouche Nunes et al. (2019), Lorenz et al. (2019) at a ratio of 65:20:15. To make it a random sampling approach, a fixed random seed (random_state = 42) was used to make the sampling procedure deterministic, meaning each run produces the same split. The specific value 42 is an arbitrary choice and has no effect on results beyond determinism. This is important for reliable model evaluation, comparison between experiments, and debugging. The splitting was split into two stages:

- **Primary Split** – The merged dataset was split into a 65% for Training and 35% for a Secondary set.
- **Secondary Split** – The secondary set of data was split to 57.14% and 42.86% to get the overall split of 20% for validation and 15% for testing.

Table 3.6: Dataset Partitioning (65:20:15 Split)

Subset	**Split %**	**Sample Count**
Training	65%	590,617
Validation	20%	181,729
Testing	15%	136,297

3.4.2 Justification of Split Ratios

While an 80:20 ratio between training and testing is common (Talukder et al. 2025, Zhan et al. 2025), A higher validation set of data at 20% was used for more accurate estimates of model performance during training. This can allow for better hyperparameter tuning so the model can be tweaked to optimise performance on the unseen test data. This specific split has been used among other deep learning studies that require rigorous hyperparameter

tuning, such as in medical image analysis (Arouche Nunes et al. 2019) and environmental prediction modelling (Lorenz et al. 2019). A large allowance to the validation set reduces the risk of overfitting before the final evaluation on the unseen test data.

Chapter 4

Model Design and Implementation

4.1 Model Overview

4.1.1 Task Definition

The objective of this work is to address the DTI prediction problem as a binary classification task.

4.1.2 Model Inputs and Outputs

The inputs and outputs were explained in detail in my data chapter 3, but I will briefly go over the inputs and outputs. The inputs going into the model are Protein embeddings and Drug embeddings with strong semantic meaning. The output is either a yes or no classification determined by thresholds mentioned in Chapter 3.

4.1.3 Learning Paradigm

The learning paradigm, how the model learns from the data, is supervised binary classification using PyTorch-based gradient optimisation using Adam (Kingma & Ba 2015). The model learns to distinguish between binding and non-binding drug-target pairs based on labelled training data.

4.2 Problem Formulation

The prediction task is formulated as a supervised binary classification problem, where the goal is to conclude whether a given drug and protein pair produce a binding interaction. The data instances of drug molecules and target proteins are represented as fixed length numerical vectors where $\mathbf{d} \in \mathbb{R}^m$ denotes the representation of a drug molecule and $\mathbf{p} \in \mathbb{R}^n$ denotes the representation of a target protein.

The goal of the model is to learn a function $f : \mathbb{R}^m \times \mathbb{R}^n \to [0, 1]$, which maps a drug–protein pair $(\mathbf{d}, \mathbf{p})$ to a continuous interaction score $\hat{y}$ representing the predicted probability of binding: $\hat{y} = f(\mathbf{d}, \mathbf{p})$. To obtain a binary interaction label, the predicted score $\hat{y}$ is compared to the predefined decision threshold τ, which is a tunable parameter; the value of $\tau = 0.3$ is justified through data sensitivity analysis in Chapter 6:

$$\hat{y}_{\text{class}} = \begin{cases} 1, & \text{if } \hat{y} \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

The ground-truth labels $y \in \{0, 1\}$ are derived from experimentally measured binding affinity values and binarised using affinity thresholds described in Chapter 3. Pairs exceeding the binding threshold are labelled as interacting, while pairs below the non-binding threshold are labelled as non-interacting.

Classification was selected over regression to improve robustness against experimental noise and to focus learning on more definitive binding interactions. This formulation aligns with the intended application of the system, where the primary objective is to determine whether a candidate drug is likely to bind a given target rather than to predict an exact affinity value.

The learning objective is, therefore, to minimise the discrepancy between the predicted interaction labels $\hat{y}$ and the true labels y in the training dataset, allowing the model to generalise to the unseen drug-target pairs.

4.3 Integration of Input Representations

Drug to Target interaction predictions requires the combination of two different data styles, drugs being small chemical structures and proteins being amino acid sequences. These representations differ in scale, semantic meaning, and dimensionality, making direct integration inappropriate.

To address this, the model uses a dual-input design, where drugs and proteins are processed independently through dedicated encoding pathways before being combined at a later stage once they become compatible. Allowing independent processing preserved the features

and semantic meaning for each drug and protein, and joint learning is enabled once combined.

4.3.1 Drug Representation

Drug molecules are represented using MACCS fingerprints, which encode the presence or absence of predefined chemical substructures as a fixed length binary vector of 167 dimensions.

The fixed dimensionality makes it well suited for neural networks as it allows the model to learn patterns between specific substructures and binding behaviour. This also makes it computationally efficient and reproducible across multiple experiments.

4.3.2 Protein Representation

Target proteins are represented using contextual embeddings created by the pretrained ProtBERT transformer model (Elnaggar et al. 2020). Unlike fixed sequence-based embeddings, transformer-based embeddings capture bidirectional and context-dependent semantic meanings within amino acid sequences, capturing biochemical properties and structural information.

Each protein sequence is mapped to a fixed-length vector through mean pooling to produce a representation suitable for neural network processing; this is the aggregation strategy used in the original ProTrans work (Elnaggar et al. 2020). Mean pooling provides a simple method for aggregating variable-length protein sequences while preserving global contextual information learned by the transformer. This approach enables the model to leverage rich semantic information learned from large protein databases, improving generalisation to unseen targets.

4.4 Model Architecture

Due to the heterogeneous nature of Dual Inputs, the architecture is slightly modified from traditional neural networks. Separate encoding pathways are used to process each input to a usable, unified representation for interaction prediction.

The `DrugEncoder` and `ProteinEncoder` are trainable multilayer perceptrons that project the precomputed MACCS and ProtBERT features into a shared 128-dimensional latent space. Their parameters are updated jointly with the `InteractionPredictor` during training. This two-stage design separates fixed, domain-informed feature extraction from learned task-specific adaptation: the representations contribute chemical and biological data, while

the encoders learn to emphasise the most informative components of those representations for binding prediction.

4.4.1 Drug Encoder

The drug encoder is responsible for transforming molecular SMILES string into a numerical representation using MACCS (Molecular ACCess System) keys (Durant et al. 2002). The SMILES representation is parsed into an RDKit object, where MACCS fingerprint generation is applied, producing a 167-bit binary vector.

This representation captures essential structural properties of small molecules while maintaining a compact and consistent dimensionality. In cases where a SMILES string cannot be successfully parsed, a zero vector is returned to preserve input dimensional consistency and prevent pipeline failure.

4.4.2 Protein Representation

Protein sequences are represented using contextual embeddings derived from a pretrained ProtBERT transformer model (Elnaggar et al. 2020). Each amino acid sequence is tokenised and passed through the frozen ProtBERT model to obtain per-token hidden states. Mean pooling is then applied across the sequence dimension of the final hidden layer, producing a single 1024-dimensional embedding that summarises the biochemical and contextual properties of the entire sequence.

This process is computationally expensive, so each embedding is cached locally using hash-based lookup, reducing overhead when a sequence has already been computed. This is particularly effective during dataset re-processing, as previously computed embeddings are loaded directly from the disk.

4.4.3 Protein Encoder

The protein encoder is a four-layer MLP (1024→512→256→128) that compresses the precomputed ProtBERT embedding into a 128-dimensional latent representation. This follows the design pattern used by Singh et al. (2023), where frozen pretrained protein language model embeddings are transformed into a lower-dimensional shared latent space by a learned nonlinear projection. The layer widths being halved at each step, acts as a soft information bottleneck that encourages the encoder to only keep components of the ProtBERT embedding that are most informative for interaction prediction. Unlike the frozen ProtBERT model, this encoder is trainable and participates in gradient-based optimisation during model training.

4.4.4 Prediction Network

The prediction network contains four hidden layers (256, 128, 64, 1) using ReLU activations for learning non-linear interactions between the drug and protein representations. The network also makes use of dropout and L2 regularisation to prevent overfitting, which is a common issue in models with limited training data and high-dimensional inputs. The sigmoid is applied only at inference, to map the raw logit to a probability. During training, BCEWithLogitsLoss applies sigmoid internally when computing the loss for numerical stability. Here is a table describing the architecture of the prediction network:

Table 4.1: Model Architecture and Training Hyperparameters

Parameter	**Value**
Architecture	
Drug encoder	167→256→128
Protein encoder	1024→512→256→128
Predictor	256→256→128→64→1
Activation	ReLU
Batch normalisation	Yes
Dropout	0.4
Training	
Loss function	BCEWithLogitsLoss
Optimiser	Adam
Learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Batch size	512
Max epochs	100
Regularisation	
Early stopping	Yes (patience = 15)
LR scheduler	CosineAnnealingLR ($T_{\max} = 100$, $\eta_{\min} = 1 \times 10^{-5}$)

The full dual-branch architecture is defined as three PyTorch `nn.Module` classes: `DrugEncoder`, `ProteinEncoder`, and `InteractionPredictor`, composed inside the top-level `DTI_DNN` module. Rather than applying a uniform dropout rate throughout the network, a tapered schedule was used in which deeper layers receive lower dropout than earlier ones. The reasoning is that deeper representations are more abstract and less redundant, so aggressive dropout at that stage risks destroying genuinely useful features rather than preventing

the co-adaptation that dropout is designed to address. A two-thirds multiplier (≈ 0.67) was chosen empirically: with a base rate of 0.4, the deeper dropout layers therefore use approximately 0.27 (0.4×0.67). Stronger tapers underused the regularisation budget in deeper layers, while weaker tapers produced less stable training; the 0.67 value retained the benefit of regularisation without visible degradation during the iterative experiments described in Chapter 6. The forward-pass definition of the top-level module is shown below.

```
class DTI_DNN(nn.Module):
    def __init__(self, drug_input_dim=167, prot_input_dim=1024,
                 drug_hidden=256, prot_hiddens=(512, 256),
                 encoder_out=128, predictor_drop=0.4, encoder_drop
                     =0.4):
        super().__init__()
        self.drug_encoder = DrugEncoder(drug_input_dim,
            drug_hidden,
                                        encoder_out, encoder_drop)
        self.prot_encoder = ProteinEncoder(prot_input_dim,
            prot_hiddens,
                                           encoder_out,
                                               encoder_drop)
        self.predictor = InteractionPredictor(encoder_out * 2,
            predictor_drop)

    def forward(self, x_drug, x_prot):
        drug_repr = self.drug_encoder(x_drug)
        prot_repr = self.prot_encoder(x_prot)
        fused = torch.cat([drug_repr, prot_repr], dim=1)
        return self.predictor(fused)
```

Listing 4.1: dual-branch forward pass in DTI_DNN

4.4.5 Fusion and Prediction Layer

The design choices reflected here were informed through iterative experimentation, the full progression is documented in Chapter 6.

Once both inputs have been encoded, both representations are combined to form a joint input for the prediction model. The binary MACCS fingerprint with dimensionality 167 and the ProtBERT embedding with dimensionality 1024 are individually encoded to have dimensionality of 128 before being concatenated into a single vector with dimensionality 256 representing the drug-target pair.

This combined vector is then passed through the prediction network described in Table 4.1 and produces a binary classification based on the optimal threshold, $\tau = 0.3$, determined through experimentation.

4.5 Training Procedure

This section describes how the model parameters are optimised during training, a process that is important to how the model learns meaningful patterns and builds internal representations on the data it's exposed to.

4.5.1 Loss Function

The model is trained using Sigmoid + Binary Cross Entropy (BCE) loss function to have BCEWithLogitsLoss, defined as:

$$E = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right],$$

Where N is the number of training samples, y_i is the true binary label for the i-th sample, and $\hat{y}_i$ is the predicted probability of the positive class (binding interaction) for the i-th sample. This loss function is more appropriate for binary classification because it penalises confident wrong predictions more heavily than MSE (Goodfellow et al. 2016).

To address the class imbalance identified in Chapter 3 (∼61% binders, ∼39% non-binders), the loss is weighted using PyTorch's `pos_weight` argument. This scales the positive-class contribution to the loss by the ratio of non-binders to binders computed on the training split, effectively giving underrepresented examples proportionally more influence during gradient updates without resampling the dataset.

```
n_pos = y_train.sum().item()
n_neg = len(y_train) - n_pos
pos_weight = torch.tensor([n_neg / n_pos], dtype=torch.float32).to
    (device)
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
```

Listing 4.2: class-imbalance weighting for BCEWithLogitsLoss

4.5.2 Optimisation Strategy

Model parameters were initially optimised using batch gradient descent with backpropagation. Following iterative experimentation documented in Chapter 6, the model was reimplemented in PyTorch using the Adam optimiser with mini-batch gradient descent and He

initialisation for weight initialisation. Adam is an adaptive learning rate optimiser that maintains per-parameter learning rates using a combination of momentum and RMSProp, which can lead to faster convergence and improved performance compared to traditional stochastic gradient descent (SGD) (Kingma & Ba 2015). Parameters are updated via:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t \tag{4.1}$$

Where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates for the first and second moments of the gradients, α is the learning rate, and ϵ is a small constant to prevent division by zero. Following the default hyperparameters recommended by (Kingma & Ba 2015), this work uses $\alpha = 0.001$, $\beta_1 = 0.9$ $\beta_2 = 0.999$ and $\epsilon = 1 \times 10^{-8}$, with an L2 regularisation penalty applied via weight decay (see Table 4.1). Mini-batch gradient descent is used to balance the computational efficiency of batch gradient descent with the noise reduction benefits of stochastic gradient descent, allowing for more stable convergence and better generalisation (Masters & Luschi 2018). He initialisation is used to set the initial weights of the network, which helps to mitigate issues of vanishing or exploding gradients in deep networks, particularly when using ReLU activations (He et al. 2015).

4.5.3 Hyperparameter Selection

As shown in Table 4.1, we are using mostly defaults set by Adam optimiser. The learning rate was set to 1×10^{-3}, which is a common default for Adam and provides a good balance between convergence speed and stability. Weight decay adds a penalty proportional to the squared magnitude of the weights to the loss function, which helps to prevent overfitting by adding a penalty on large weights. This encourages the model to learn simpler patterns that generalise better to unseen data (Krogh & Hertz 1992). A value of 1×10^{-4} was chosen through tuning, and it is consistent as a default value commonly used in PyTorch training pipelines. The batch size of 512 was chosen to provide a good balance between computational efficiency and stable gradient estimation. This sits at the upper boundary of the small-batch regime identified by Keskar et al. (2017), which is typically 32–512 and generalises better than larger batches.

4.5.4 Regularisation

The Dropout rate was determined through experimentation as seen in Chapter 6, and was set to 0.4, which means that during training, 40% of the neurons in the hidden layers are randomly deactivated in each iteration, helping to prevent overfitting by encouraging the model to learn more robust features that are not reliant on specific neurons (Srivastava et al. 2014). L2 regularisation is applied through the weight decay parameter in the Adam

optimiser, which adds a penalty proportional to the square of the magnitude of the weights, encouraging smaller weights and thus simpler models that are less likely to overfit.

4.6 Model Output Interpretation

The model produces a single raw logit for each drug-target pair. During training, `BCEWithLogitsLoss` applies the sigmoid function internally when computing the loss, avoiding numerical instability associated with a separate sigmoid layer. At inference time, the sigmoid function is applied explicitly in the prediction service to map the logit to a probability $\hat{y} \in (0, 1)$, representing the likelihood of binding (1) or non-binding (0).

To get a discrete prediction from the continuous output score, the score is compared against some predefined thresholds to optimise the performance of the model. We settled on a threshold of 0.3 after experimentation as it produced the highest F1 score on the test set, which means it is the optimal balance between precision and recall.

The final prediction should be interpreted as a probability instead of a definitive prediction. Therefore, this model is intended for the early-stage screening of drug-target pairs instead of replacing experimental validation.

4.7 Implementation Details

The model was originally implemented in Python using NumPy for the numerical computations. All the forward and backward propagation steps were implemented manually for full transparency over the learning process and better insight into how changes affect the model's performance. However, after conducting experiments and evaluating each improvement, the optimal implementation utilised PyTorch's framework. The dual-branch encoder design was a PyTorch-specific implementation because of the ease of use and flexibility it provides for building complex architectures and handling heterogeneous data.

Chapter 5

System Interface and API

This chapter describes the design and implementation of the system interface and API for the Drug-Target interactive Predictor. It covers the architecture of the system, the design of the backend API, the frontend design, and the integration between the frontend and backend components. Additionally, it discusses performance considerations, security measures, limitations, and provides a summary of the interface design.

5.1 System Architecture

The system architecture was designed with consideration of performance and speed, as well as an intuitive user interface. The architecture consists of a backend that contains data and the predictive model, a frontend interface that allows users to interact with the system and visualise results, and an API to link it all together.

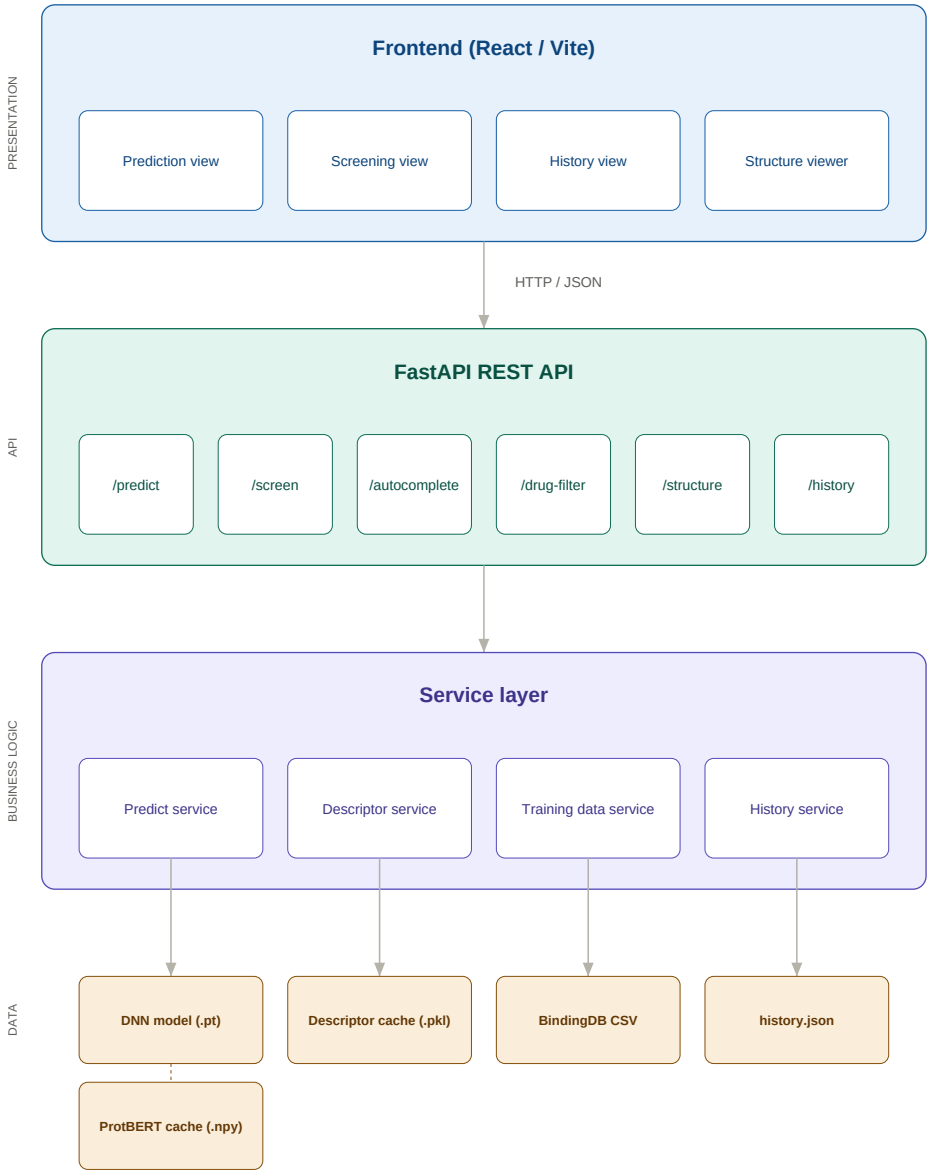

Figure 5.1: System architecture of the Drug–Target Interaction Predictor.

5.2 Backend API Design

To link the frontend and backend components, a RESTful API was designed to ensure seamless communication between the two. The API was designed with a focus on simplicity, scalability, and maintainability, allowing for easy integration with the frontend and potential future extensions.

5.2.1 Framework and Architecture

We used FastAPI as the framework for our backend API due to its high performance, ease of use, and robustness (Ramírez 2019). The API follows a modular architecture, with separate modules for data loading, model inference, virtual screening, and input validation. This modular design allows for better organisation of code and easier maintenance.

5.2.2 Endpoint Specification

The API exposes twelve endpoints, grouped by domain in Table 5.1. The Prediction and screening endpoints handle model inference, and the autocomplete and filtering endpoints support the search and drug selection workflows. The history endpoints allow the user review and clear previous predictions. The structure endpoint provides on-demand molecular visualisations provided by RDKit.

Table 5.1: API endpoint specification.

Method	Path	Description
Prediction		
POST	`/predict`	Accepts a single drug–target pair and returns a binding probability score.
POST	`/screen`	Accepts up to 100 drugs against a single protein target and returns ranked scores (virtual screening).
Autocomplete and Filtering		
GET	`/autocomplete/drug`	Returns up to 10 drug matches by name or SMILES substring.
GET	`/autocomplete/protein`	Returns up to 10 matching protein target names.
GET	`/autocomplete/drug-sample`	Returns a random sample of drugs for quick screening.
GET	`/drug-filters`	Returns available filter definitions for the frontend to render.
GET	`/drug-filter`	Returns compounds matching physicochemical and activity filters.
GET	`/drug-filter/count`	Returns the count of compounds matching the given filters.
Visualisation		
GET	`/structure/svg`	Returns an Scalable Vector Graphic (SVG) depiction of a molecule from its SMILES string.
History		
GET	`/history`	Returns the most recent 100 predictions and screens.
DELETE	`/history`	Clears all stored prediction history.
System		
GET	`/health`	Returns the health status of the API.

5.2.3 Data Loading and Preprocessing

At startup time, the backend performs several data loading and preprocessing steps to ensure that the system is ready for user interactions.

The full BindingDB CSV containing drug SMILES strings, protein target names, amino

acid sequences, and binary interaction labels, is loaded into a pandas DataFrame at module import time by the training data service. From this, the autocomplete endpoint creates drug and target lists.

Molecular descriptors for all unique compounds are computed using RDKit (RDKit Contributors 2025), and these are persisted to a pickle file so that subsequent startups can load them directly rather than recomputing them. A staleness check compares the cached descriptor count against the number of unique SMILES in the current dataset, triggering a rebuild if they diverge by more than five per cent. This was chosen to tolerate small dataset changes to ensure cached data is consistent with the actual dataset. This is shown in the following code snippet:

```
def _load_or_build() -> pd.DataFrame:
    if os.path.exists(_CACHE_PATH):
        cached = pd.read_pickle(_CACHE_PATH)
        n_unique = DRUG_ROWS["drug_smiles"].nunique()
        if abs(len(cached) - n_unique) < n_unique * 0.05:
            return cached
        print("Descriptor cache is stale, rebuilding...")

    table = _build_descriptor_table()
    os.makedirs(_CACHE_DIR, exist_ok=True)
    table.to_pickle(_CACHE_PATH)
    return table
```

Listing 5.1: Descriptor cache loading with staleness check.

Protein representations are handled through a hash-indexed cache of precomputed ProtBERT embeddings (Elnaggar et al. 2020). Each protein sequence is mapped to a 1024-dimensional embedding vector stored as a NumPy `.npy` file, keyed by an MD5 hash of the sequence. At startup the prediction service builds a lookup table to map protein names to their amino acid sequences, reducing retrieval time of embeddings.

To support the known-binders filter, a binder index is constructed by grouping together the training data by target name and collecting the set of SMILES strings that bind.

5.2.4 Model Inference Pipeline

This section describes how raw drug and target inputs become a binding probability score through feature extraction and neural network inference.

Model Artefact

The trained model is stored as a `.pt` file, which is PyTorch's standard serialisation format for model weights. During training, the training script monitors the validation AUC after each epoch and saves the model's `state_dict` — a Python dictionary mapping each layer name to its learned weight tensor — whenever a new best validation score is achieved. This best-checkpoint strategy ensures that the saved file captures the model at its best point rather than just at the end of training where overfitting may have occurred.

The resulting file (`original_best.pt`) contains only the learned parameters (weights and biases) of the `DTI_DNN` network, not the architecture definition itself. This means the model class must be available in code at load time so that PyTorch can reconstruct the network and populate it with the saved weights.

Model Loading at Startup

When the FastAPI backend starts, the prediction service loads the `.pt` file once and retains the model in memory for the lifetime of the process. The loading procedure is as follows:

1. An empty `DTI_DNN` instance is created with the same architecture hyperparameters used during training: a 167-dimensional drug input passed through a drug encoder (167→256→128), a 1024-dimensional protein input passed through a protein encoder (1024→512→256→128), and the resulting 256-dimensional representation passed through the interaction predictor (256→256→128→64→1), all with a dropout rate of 0.4.

2. `torch.load` deserialises the `.pt` file into a state dictionary, mapping it to the best available hardware accelerator (Apple MPS, CUDA, or CPU).

3. `load_state_dict` populates the empty model with the saved weights.

4. The model is set to evaluation mode via `model.eval()`, which disables dropout and batch normalisation updates so that inference results are deterministic.

Because the model is loaded once at startup rather than on each request, inference latency is kept low and repeated file I/O is avoided.

Feature Extraction and Forward Pass

The two feature vectors (SMILES, Amino acid sequence) are passed through their respective encoder subnetworks and joined into a single joint representation. This joint vector is then fed through the interaction predictor head, which outputs a probability score between 0 and 1, where values above 0.3 indicate predicted binding.

Batch Inference Pipeline

To fully emulate virtual screening, the pipeline also supports batch predictions.

5.2.5 Virtual Screening Pipeline

Filtering

The virtual screening pipeline allows users to test multiple drugs against a single protein through a series of filters. Users can apply filters on physiochemical properties such as:

Table 5.2: Available compound filters for virtual screening.

Filter	Description
Molecular Weight Range	Molecular weight in Daltons (Lipinski et al. 2001, Veber et al. 2002).
Ring Count	Filters by the number of ring structures present in the molecule (Bemis & Murcko 1996).
Lipinski's Rule of Five	Enforces MW $\leq$ 500, LogP $\leq$ 5, HBA $\leq$ 10, and HBD $\leq$ 5, selecting compounds with drug-like oral bioavailability (Lipinski et al. 2001).
Known Binders	Restricts the pool to compounds with a positive interaction label against a user-selected protein target in the training data.

These filters are applied as boolean masks over a precomputed descriptor table to narrow down large compound sets efficiently.

Limited Selection

When the number of compounds matching the filters exceeds the processable limit, the system applies the *MaxMin* diversity-picking algorithm using MACCS-based Tanimoto distance (Bajusz et al. 2015). Instead of using a random sample, this algorithm makes sure to select a structurally diverse subset to ensure broad coverage of drugs in the test. The selected drug molecules are scored against the target protein using the batch inference pipeline, ranked by binding probability, and matched to metadata such as molecular weight, LogP, and ring count before being returned to the frontend.

5.2.6 Input Validation and Error Handling

Input Validation

Validating the inputs is handled through two complementary mechanisms. At the schema level, all requests and responses are defined as Pydantic models, which enforce type constraints and use automatic validation of incoming JSON requests (Colvin 2019).

For query parameter endpoints, FastAPI's `Query` annotations are used to enforce value constraints directly in the function signatures. This is allowing us to limit the batch size to 100, capping the drug sample endpoint to 50 results, and keeping the Scalable Vector Graphics (SVG) rendering dimensions between 50 and 500 pixels. FastAPI rejects requests that violate these constraints automatically with 422 unprocessable entity response before reaching the service layer.

Error Handling

The service layer will raise `descriptive exceptions` when errors occur during inference. These are caught at the route level and returned as HTTP error responses using FastAPI's `HTTPException`. Status codes distinguish the differences between client errors (400 for bad input) and server errors (500 for unexpected failures).

5.3 Frontend Design

Creating a simple and intuitive design is essential for usability, we utilised a React framework to ensure speed and optimisations for what we need it to achieve.

5.3.1 Frontend Framework

To develop the frontend, we have used Gatsby, a React framework that is optimal for static websites. It does this by pre-rendering pages into HTML, called Static Site Generation (SSG), resulting in superior performance. Gatsby benefits from high security with no server-side attacks as it doesn't have a server-side database or dependencies, only static files (So 2021, Gatsby, Inc. 2024).

5.3.2 User Interface Design

To keep things simple, we only have two tabs for the user to test a single drug to a single protein target, and batch virtual screening to emulate a wet-lab experiment more closely.

Ethical Considerations Message

To inform the user of appropriate use, we have a clear message regarding the ethical considerations of the product to prevent misuse and over reliance on results produced.

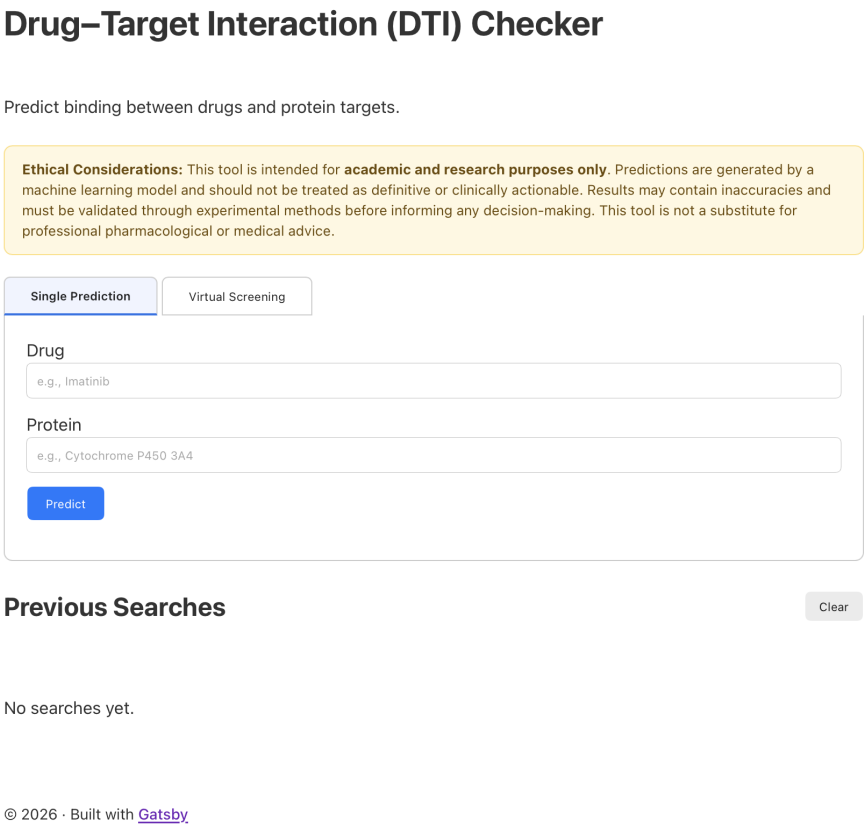

Figure 5.2: The web page at initial startup with an ethical considerations message

Single comparison

One input box for the molecule input and another for target input, both have suggested activators greyed out, and once the user begins typing, a dropdown menu will present the closest match to what's been written in a top-down fashion.

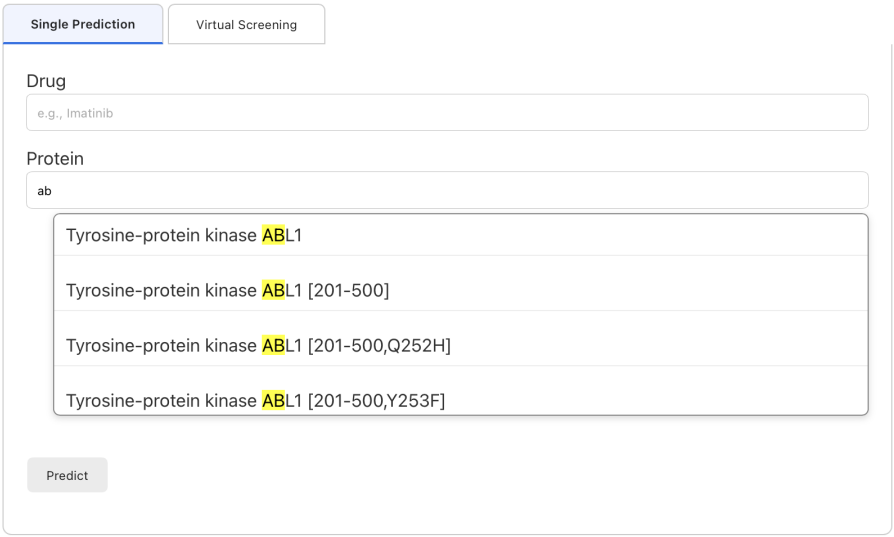

Figure 5.3: Single prediction input with autocomplete dropdown.

Once selected and compared, the results appear just below in a simplified form:

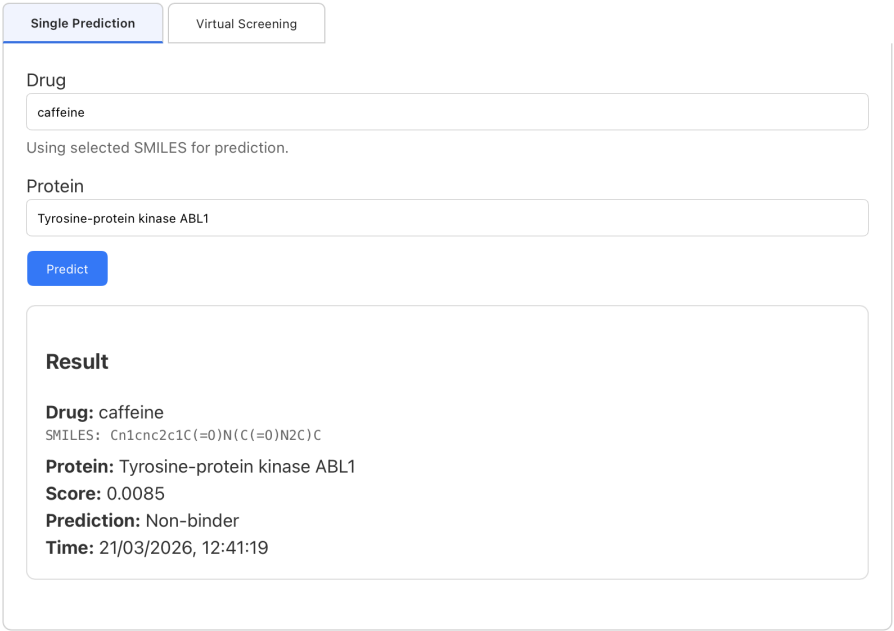

Figure 5.4: Single prediction result displaying the binding probability and classification.

Below this tab is the prediction history, showcasing previous comparisons made by the

user, sorted by the most recent.

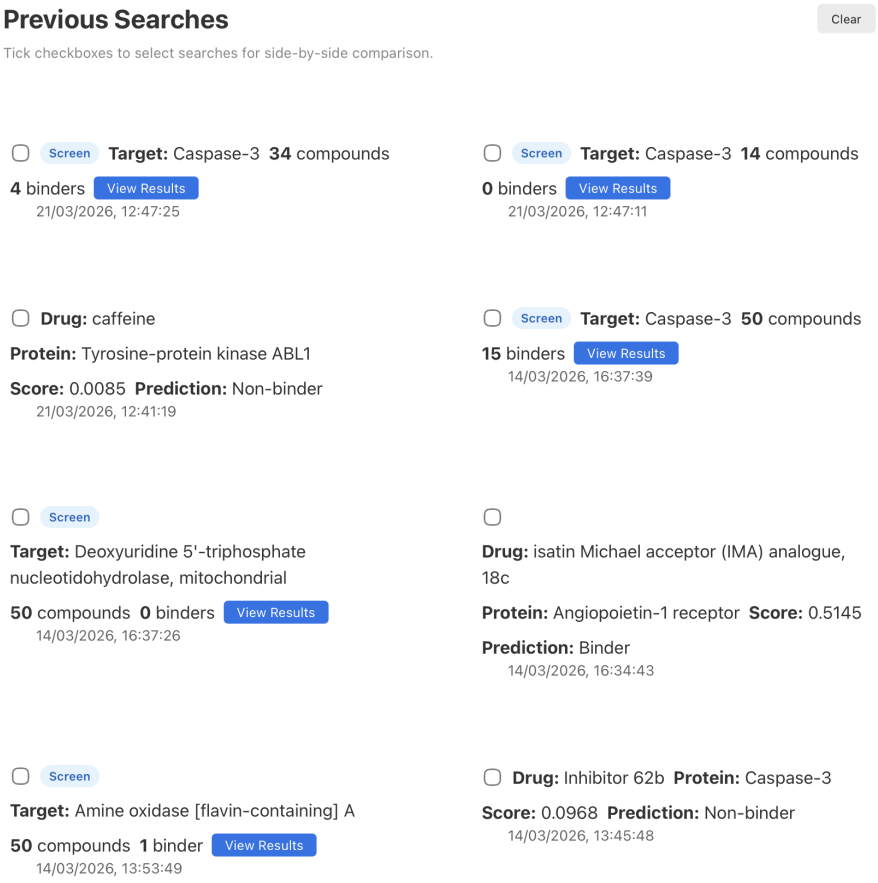

Figure 5.5: Prediction history panel showing previous queries sorted by date.

Batch Comparison

To begin the batch comparison, the user can filter for the group of molecules they'd like to compare against a protein. The user can toggle for Lipinski's rule of five, select a preset for Molecular Weight Range and number of Rings.

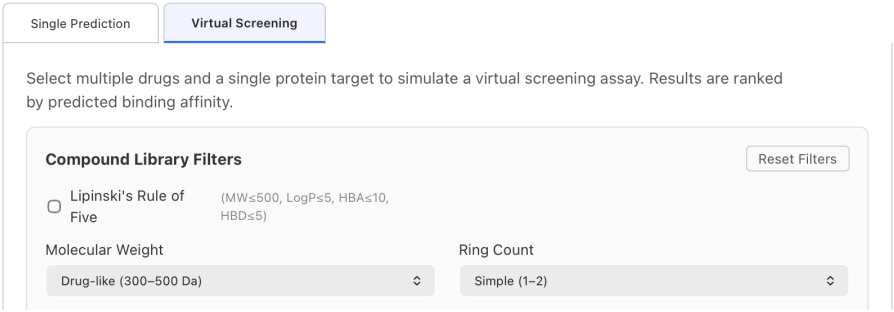

Figure 5.6: Virtual screening filter controls for physicochemical properties.

We also allow the user to filter molecules that bind to a specific target to see if there are any crossovers/they're more likely to bind. At the bottom of the filter section, it will instantly update the number of molecules that match the set of filters applied.

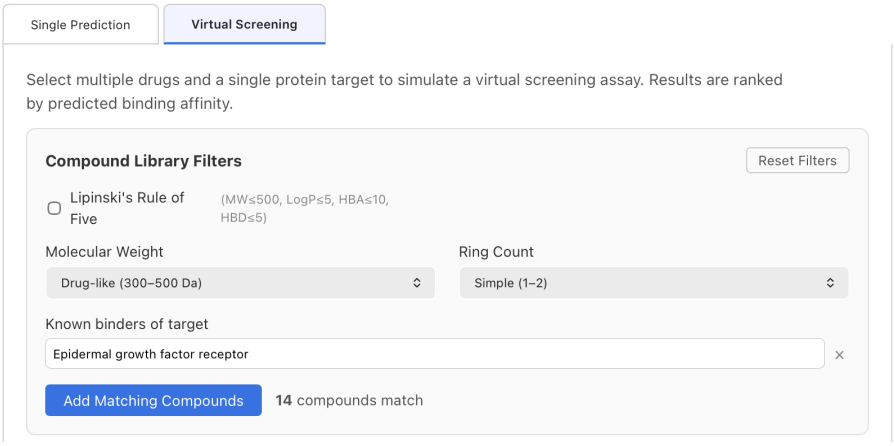

Figure 5.7: Complete filter panel with known-binders target selection and live compound count.

Once the user has added up to 100 molecules for testing and selected the target, they can continue with testing. This will produce a result table sorting the bindings from most likely to least likely and colour code them. The table also shows meaningful metadata for each molecule, and hovering over the molecule name will produce a widget with the chemical structure of the molecule.

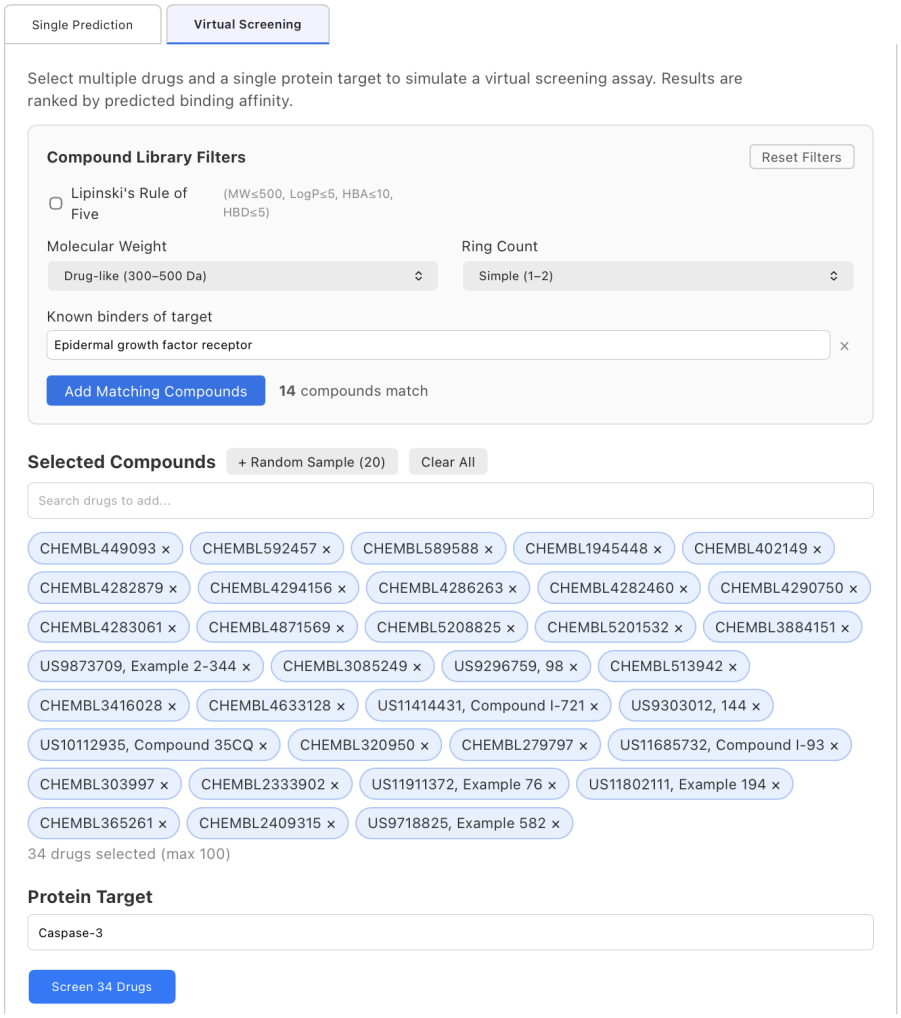

Figure 5.8: Virtual screening overview showing selected compounds and target protein.

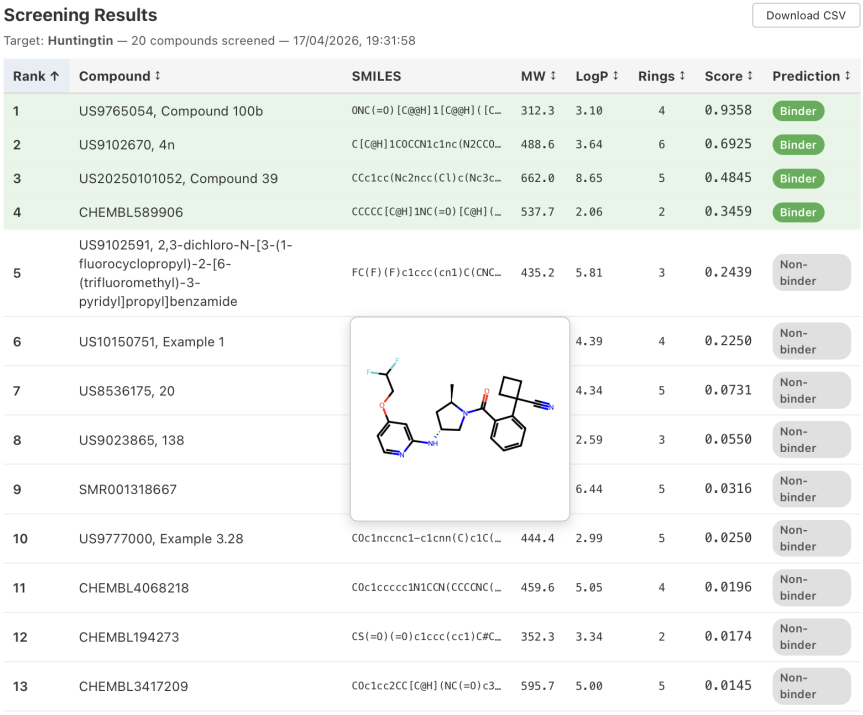

Figure 5.9: Screening results table with colour-coded binding scores and molecular structure popup.

5.3.3 Autocomplete and Search Functionality

Autocomplete was a necessary feature to help users quickly identify the activators in question. Some have names with over 15 characters, so the autocomplete search makes it more usable for people.

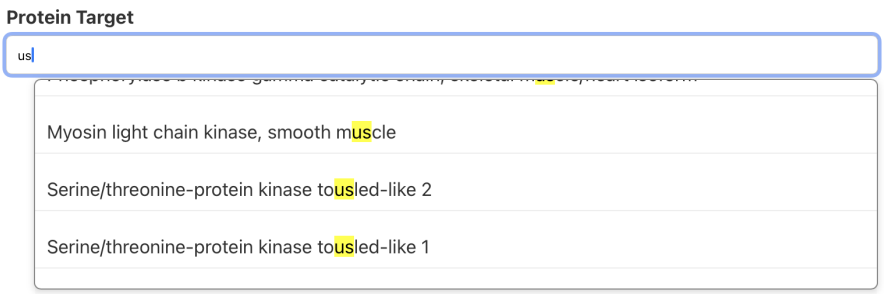

Figure 5.10: Autocomplete handling long compound names.

Search is linked to the bindingDB database to have the most up-to-date naming for

molecules and targets.

5.3.4 Previous Searches

An important design feature is how the previous searches are visualised. We have it sorted by date and time occurred, with the ability to view results. This allows the user to potentially compare results between closely related targets and drug molecules. We decided to display the previous results as a widget if the user would like to see the information closer.

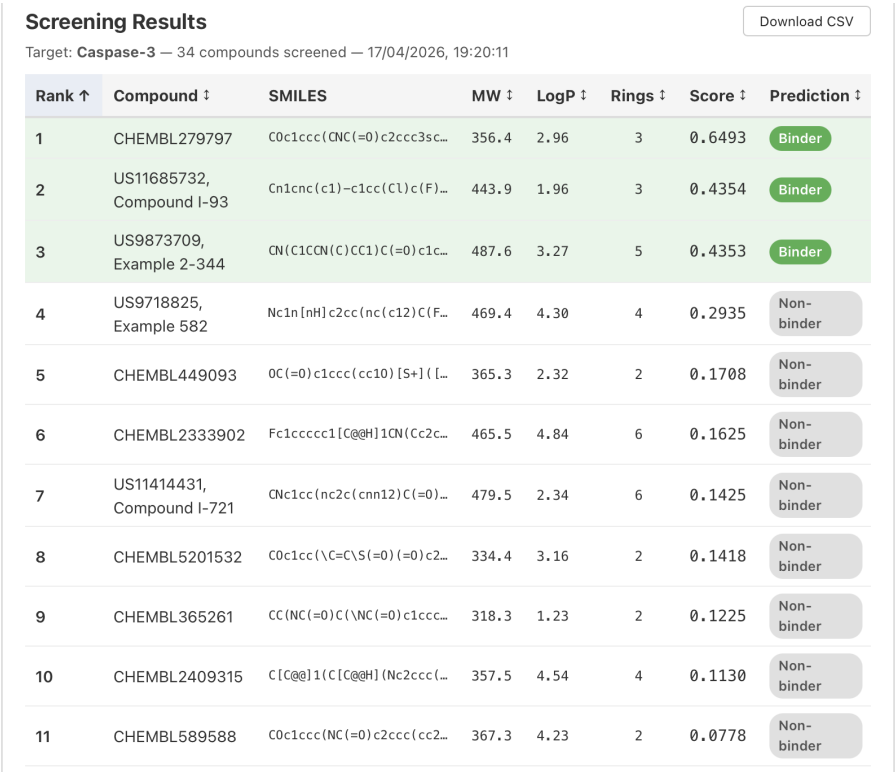

Figure 5.11: Expanded results widget showing detailed screening output.

Comparisons

Allowing the user to compare previous results is extremely important and can highlight certain differences between two near identical searches, extremely valuable in drug discovery. This provides the user with insightful information into their tests which can hopefully improve the usability of the product. The comparison can also be between batch screening and single screening, allowing for full customisable testing.

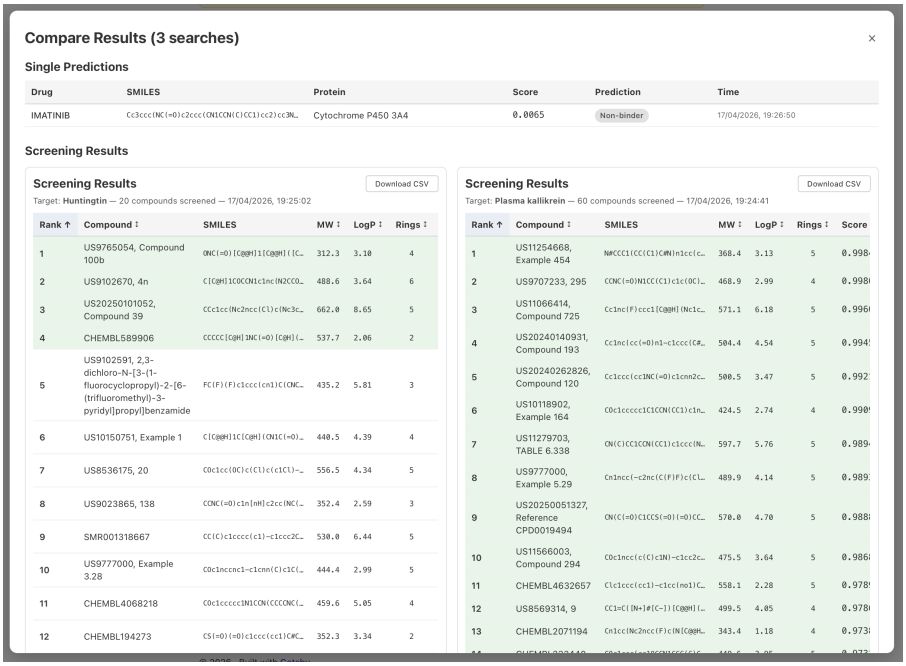

Figure 5.12: Side-by-side comparison of three previous prediction results.

5.4 Frontend-Backend Integration

The frontend and backend communicate through HTTP requests to the RESTful API described in Section 5.2. All the data transmissions use JSON, where the frontend sends user inputs and receives structured responses containing prediction scores, screening results, or autocomplete suggestions.

For the autocomplete, endpoints are called on each keystroke with a debounced delay to return up to 10 matches to populate dropdown suggestions. When predicting a single pair, the frontend issues a POST request to `/predict` and displays the returned score and binding classification. For full virtual screening, the frontend initially retrieves the filtered molecules set from `/drug-filter` endpoint, then submits the selected drugs to `/screen` for batch scoring. Molecular structure diagrams are rendered by requesting Scalable Vector Graphics (SVGs) from the `/structure/svg` endpoint and embedding them directly in the interface as a widget.

Prediction history is fetched from `/history` on page load and refreshes after each new prediction.

5.5 Performance Considerations

The design choices were made to minimise response times during use, for a better user experience.

The most significant improvement is the computations at startup described in Section 5.2.3. By loading the training dataset, computing molecular descriptors, and indexing known binders at server startup, the system avoids expensive computations during inference. ProtBERT embeddings, which are the most costly feature to compute, are precomputed offline and cached as individual `.npy` files, reducing protein featurisation to a single file read.

To allow for virtual screening, we run all candidate drugs through the model in a single forward pass using batch inference. This will exploit CPU or GPU parallelism through PyTorch's tensor operations (Paszke et al. 2019).

5.6 Security and Robustness

The frontend is a static web page that is independent of the backend, so no server-side rendering, therefore making server-side attacks redundant.

The backend validates all incoming data through Pydantic schemas and FastAPI query parameter constraints, rejecting improper requests before they reach the service layer. This provides a degree of protection against injection attacks, as inputs are type-checked and bound-checked before being passed to RDKit or the model.

At the moment there isn't any user authentication or account system or sensitive data, heavily reducing the attack surface area. Therefore, upon deployment the prediction history will be shared among all users. If the system were to be deployed in a multi-user clinical setting, user isolation through accounts and authentication will need to be added.

5.7 Limitations

The current system has some limitations that must be acknowledged. There is currently no session management or accounts system, meaning the history section is global instead of for a user. But this is okay in its current form, as it's just a prototype on how the model can be accessed in an intuitive manner.

The model currently just produces a binding probability, but it doesn't offer a reason into why the interaction is predicted to occur. An insight into what drives the prediction, such as molecular features or substructures, would improve reliability in the system for users.

Another insight that could benefit users would be a structural visualisation of how the bound pair would look at a molecular level. Currently, users can only view the 2D struc-

ture of drug molecules, but there is no binding structure prediction. Integrating molecular docking tools can address these issues in future work.

Finally, the model has only been trained on bindingDB from a fixed dataset. There isn't an option for users to upload their own data source, limiting its ability for predictions on drugs and proteins outside the training set.

Chapter 6

Evaluation

Here we will evaluate the performance of our drug-target interaction (DTI) prediction model across multiple iterations of development. The evaluation will be structured around three key areas: **1**: model progression through iterative improvements (training, architecture, framework); **2**: sensitivity to data and decision thresholds; and **3**: comparison of the final model against established state-of-the-art DTI methods on BindingDB. Through this comprehensive evaluation, we aim to demonstrate the effectiveness of our modelling choices and identify areas for future improvement.

6.1 Evaluation Methodology

Within this section, we will outline the components of our evaluation methodology, including the metrics we will use to assess model performance, the data splits for training and testing, and the rationale behind our choice of a decision threshold for classification.

6.1.1 Metrics

To evaluate our models, we will find out the following metrics from the training, validation, and test set predictions:

- Area Under the Receiver Operating Characteristic Curve (AUC-ROC)
- Area Under the Precision-Recall Curve (PR-AUC)
- F1 Score
- Precision
- Recall (Sensitivity)

- Accuracy
- Confusion Matrix

Accuracy is a fundamental metric that measures the performance by finding out the ratio between correct predictions and all predictions. This alone would be meaningless because in an imbalanced dataset, a model could achieve high accuracy by simply predicting the majority class. Precision and recall are crucial in the context of DTI prediction, where false positives (predicting a drug-target interaction that does not exist) can lead to wasted resources in experimental validation, while false negatives (failing to predict a true interaction) can result in missed opportunities for drug discovery. The F1 score balances precision and recall into a single metric, providing a more holistic view of model performance. AUC-ROC and PR-AUC are threshold-independent metrics that summarise model performance across all possible decision thresholds. AUC-ROC captures the trade-off between the true positive rate and the false positive rate, and can be interpreted as the probability that the model assigns a higher score to a random positive instance than to a random negative one. PR-AUC instead captures the trade-off between precision and recall, which is particularly informative with class imbalance. Finally, the confusion matrix provides a detailed breakdown of true positives, false positives, true negatives, and false negatives, allowing for a granular analysis of model performance (GeeksforGeeks 2025).

6.1.2 Data Splits and Reproducibility

This work uses a fixed 65/20/15 split of the bindingDB data for our training, validation, and test sets. However, this is contrasting a 10-fold cross-validation approach used by DeepCDA and MINDG, which means that the reported metrics from those papers are not directly comparable to our results, but rather serve as approximate reference points. To address all possible corridors of evaluation, we will also adjust the data splits to assess any changes in performance.

6.1.3 Decision Threshold

Another important aspect of our evaluation is the choice of the decision threshold for classifying predictions as positive or negative interactions. We have opted for pAffinity value > 7.0 determining binder and pAffinity value < 5.3 determining non-binder with a grey area of 1.70 pAffinity units as mentioned in 6.4.2. However, to improve the robustness of our model, we will experiment with these figures to fully evaluate the model.

6.2 Model Evaluation

Here we will begin the evaluation of our models, opening with a basic NumPy implementation and iteratively improving it through changes to the loss function, architecture, and training procedure.

6.2.1 Experimental Setup

Before we venture further, it's good to determine our setup; we will run each model on the same hardware (Macbook Air M3, 16GB RAM) to ensure a fair comparison of training times and computational efficiency. We will train the NumPy iteration models with a maximum of 1000 epochs; the PyTorch model uses 100, both will introduce and implement early stopping based on validation loss to prevent overfitting and reduce unnecessary computation. We cap the NumPy to 1000 epochs to allow for sufficient convergence under full-batch gradient descent; the PyTorch model was restricted to a tenth of that because of the mini-batch gradient descent and to reduce the chance of overfitting. This provided the PyTorch model with more gradient updates per epoch, so equivalent optimisation progress is made in far fewer epochs. To keep consistency throughout, each iteration will be evaluated on the same subset of the BindingDB dataset, the class distribution of which is shown in Table 6.1. This 18,520-row subset has already had the grey area ($5.3 \leq$ pAffinity ≤ 7.0) removed during the preprocessing stage described in Chapter 3, so the iterative experiments are trained and evaluated only on high-confidence binders and non-binders.

Table 6.1: Class distribution across train, validation, and test splits.

Split	**Binders**	**Non-binders**	**Total**
Train	7,291	4,747	12,038
Validation	2,226	1,478	3,704
Test	1,671	1,107	2,778
Total	11,188	7,332	18,520

6.2.2 Baseline: Basic NumPy Model

To establish a baseline for our DTI prediction task, we implemented a simple feedforward neural network using only NumPy and a deliberately minimal baseline with a low complexity. This model consists of a single hidden layer with sigmoid activations, which is also present on the output layer, and it is trained using Binary Cross-Entropy (BCE) loss. The absence of many components intentionally allows for incremental improvement, and any improvement

in performance can be attributed to a specific change and not a combination of them. We trained the model with these settings:

Table 6.2: Hyperparameter selection in the NumPy baseline. Dashes (—) indicate features absent from the baseline model.

Parameter	**Basic (NumPy)**
Architecture	
Hidden size	32
Activation	Sigmoid
Batch normalisation	—
Dropout	—
Training	
Loss function	BCE
Optimiser	—
Learning rate	0.5
Weight decay	—
Batch size	Full batch
Max epochs	1000
Regularisation	
Early stopping	—
LR scheduler	—

After training this baseline model, we evaluated its performance against the validation and test sets. The results of key metrics are summarised in the following graph:

Performance Metrics — Basic NumPy Model

	Train	Validation	Test
Accuracy	0.7050	0.6936	0.7099
AUC-ROC	0.7565	0.7481	0.7567
PR-AUC	0.8056	0.8029	0.8058
F1	0.7597	0.7521	0.7605
Precision	0.7499	0.7318	0.7552
Recall	0.7697	0.7736	0.7660

Figure 6.1: Baseline model performance across key metrics.

The data in this table shows strong consistency across splits, which is a sign of minimal overfitting and deals well with unseen data. A recall of ∼0.76 indicates that this model is good at catching positive cases, which is important when predicting a false negative is costly. A strong recall is important for drug discovery, as missing a true drug-target interaction could mean overlooking a potential binding candidate. The AUC-ROC of ∼0.75 suggests that the model has a reasonable ability to distinguish between binders and non-binders, although there is certainly room for improvement. The Precision score of ∼0.75 indicates that when the model predicts a positive interaction, it is correct 75% of the time, which is decent but could lead to some false positives that would require experimental validation.

Hyperparameter tuning

To further evaluate the baseline model, we conducted a hyperparameter tuning experiment by varying the learning rate and hidden layer size. After doing a sweep of learning rate and hidden size comparing these values together, we retrieved these graphs comparing the difference in each parameter:

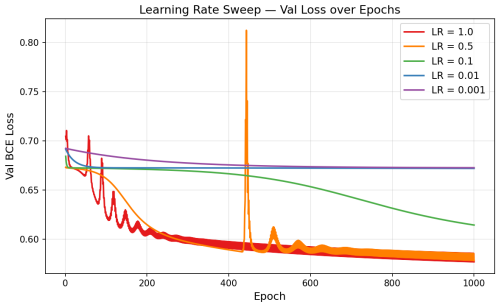

Figure 6.2: *
Learning Rate Sweep

Figure 6.3: *
Hidden Layer Size Sweep

Figure 6.4: Hyperparameter tuning results for the baseline model.

From the learning rate sweep we learned that 0.1 achieves the same result as 0.5 but with a smoother loss curve, which is a sign of more stable training. From the hidden layer size sweep we learned that accuracy and F1 score are stable across all hidden sizes and AUC-ROC only improves marginally. This suggests the basic model has reached a boundary in performance imposed by its single layer-architecture, and that further improvements will likely require architectural changes rather than just tuning hyperparameters.

6.2.3 Training Procedure advancements

After seeing how our very basic model performed, we have decided to make progressive changes to the training procedure to improve performance.

Minibatch Gradient Descent

To start with, we will add minibatch gradient descent because it can provide more stable and efficient updates compared to full batch training, especially on larger datasets. After making the change, we saw a change to the metrics shown below:

Performance Metrics — Iteration 1 (Mini-batch GD)

	Train	Validation	Test	+/- vs Basic
Accuracy	0.7163	0.7065	0.7156	+0.0058
AUC-ROC	0.7642	0.7543	0.7629	+0.0061
PR-AUC	0.8127	0.8087	0.8103	+0.0045
F1	0.7817	0.7746	0.7785	+0.0179
Precision	0.7319	0.7193	0.7325	-0.0227
Recall	0.8388	0.8392	0.8306	+0.0646

Figure 6.5: Iteration 1 model performance across key metrics.

The addition of mini-batch gradient descent improved recall substantially at the cost of a small drop in precision, reflecting the noisier gradient updates – a known trade-off that the subsequent iterations will aim to address. And a different loss curve compared to the baseline, this change is because the model is now updating its weights more frequently, which can lead to faster convergence and a smoother loss curve compared to the baseline's full batch training.

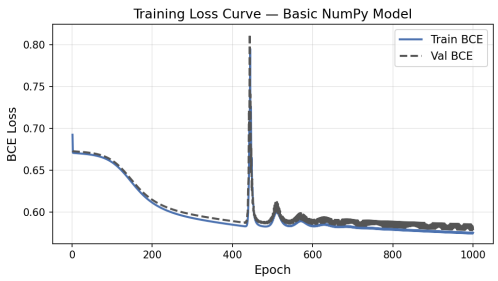

Figure 6.6: *
Baseline (Full-batch GD)

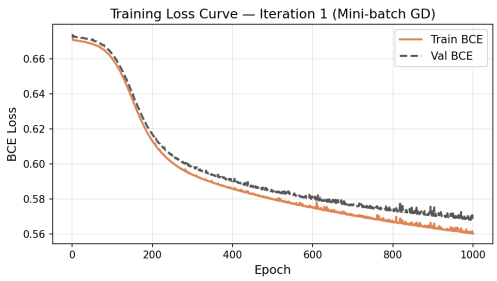

Figure 6.7: *
Iteration 1 (Mini-batch GD)

Figure 6.8: Training loss curves comparing the baseline to Iteration 1.

During the training of the baseline model, we can observe a significant instability spike. This is a characteristic of full-batch gradient descent with an aggressive learning rate. Mini-batch gradient descent eliminates this instability by providing more frequent updates, which allows the model to adjust its weights more smoothly and avoid large jumps in the loss landscape. This results in a more stable training process, as evidenced by the smoother loss curve in Iteration 1 compared to the baseline.

He initialisation

To accompany minibatch gradient descent, we will also add He initialisation to break the symmetry and prevent vanishing gradients at the start of training (He et al. 2015). He initialisation works by setting the initial weights of the network using this equation:

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right) \tag{6.1}$$

where n_{in} is the number of input units in the layer. This method is particularly effective for layers with ReLU activations, as it helps maintain the variance of the activations throughout the network, which can lead to faster convergence and improved performance. He initialisation was designed for ReLU activations specifically; applying it alongside sigmoid hidden units is a deliberate intermediate step, as ReLU hidden activations are introduced in the following iteration. Nonetheless, the variance-preserving property of He initialisation still offers an improvement over naive small weight initialisation ($\times 0.01$) regardless of activation function. These small improvements can be seen within these metrics and comparison of loss curves:

Performance Metrics — Iter 2 — He Initialisation

	Train	Validation	Test	+/- vs Iter 1
Accuracy	0.7201	0.7146	0.7214	+0.0058
AUC-ROC	0.7707	0.7598	0.7681	+0.0052
PR-AUC	0.8186	0.8134	0.8148	+0.0045
F1	0.7840	0.7808	0.7831	+0.0046
Precision	0.7359	0.7252	0.7364	+0.0040
Recall	0.8387	0.8455	0.8360	+0.0054

Figure 6.9: *
Iteration 2 model performance across key metrics.

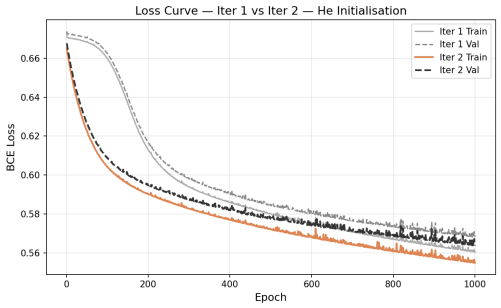

Figure 6.10: *
Training loss curves comparing Iteration 1 to Iteration 2.

Figure 6.11: Iteration 2 improvements.

The improvements from Iteration 1 to Iteration 2 are modest but consistent across all metrics.

ReLU activations

To further improve the training stability and representational capacity of our model, we will replace the sigmoid activations in the hidden layer with ReLU (Rectified Linear Unit)

activations. ReLU is defined as:

$$\text{ReLU}(x) = \max(0, x) \tag{6.2}$$

ReLU addresses the vanishing gradient problem associated with sigmoid activations, as it allows for a constant gradient for positive inputs, which helps maintain the flow of gradients during backpropagation, especially in deeper networks (Nair & Hinton 2010). This change was impactful and can be seen in the metrics and loss curves:

Performance Metrics — Iter 3 — ReLU Hidden

	Train	Validation	Test	+/- vs Iter 2
Accuracy	0.8342	0.7443	0.7649	+0.0436
AUC-ROC	0.9067	0.8097	0.8231	+0.0551
PR-AUC	0.9318	0.8547	0.8576	+0.0428
F1	0.8629	0.7897	0.8044	+0.0214
Precision	0.8640	0.7809	0.8052	+0.0687
Recall	0.8619	0.7987	0.8037	-0.0323

Figure 6.12: *
Iteration 3 model performance across key metrics.

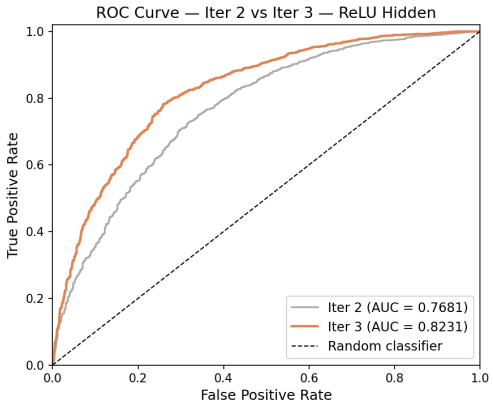

Figure 6.13: *
AUC-ROC curves comparing Iteration 2 to Iteration 3.

Figure 6.14: Iteration 3 improvements.

The switch to ReLU activations in Iteration 3 led to a noticeable improvement in all performance metrics, particularly in AUC-ROC and F1 score, which suggests that the model is now better able to capture complex patterns in the data and distinguish between binders and non-binders more effectively.

Early Stopping

Our final iteration to the training procedure is to add early stopping based on validation loss, which can help prevent overfitting and reduce training time by halting training once the model's performance on the validation set starts to degrade. After adding early stopping, we observed the following changes in performance:

Performance Metrics — Iter 4 — L2 Regularisation

	Train	Validation	Test	+/- vs Iter 3
Accuracy	0.7296	0.7149	0.7253	-0.0396
AUC-ROC	0.7785	0.7619	0.7728	-0.0503
PR-AUC	0.8278	0.8133	0.8200	-0.0376
F1	0.7937	0.7835	0.7888	-0.0156
Precision	0.7377	0.7206	0.7338	-0.0714
Recall	0.8590	0.8585	0.8528	+0.0491

Figure 6.15: *
Iteration 4 model performance across key metrics.

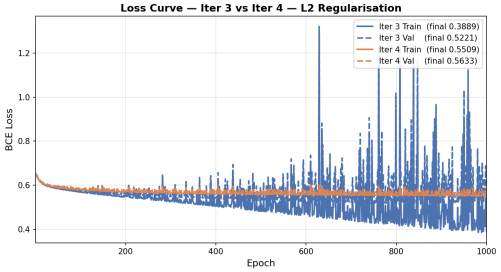

Figure 6.16: *
Training loss curves showing early stopping in Iteration 4.

Figure 6.17: Iteration 4 improvements with early stopping.

As seen in Figure 6.17, early stopping led to premature halting of training, missing out on some of the later epochs where the model could have achieved better performance. This is a common issue with early stopping, as it relies on the validation loss to determine when to stop training, and if the validation loss is noisy or has a local minimum, it can cause the training to stop too early before the model has fully converged. For these reasons we will not be using early stopping in our final model for validation loss but only for validation AUC. However, for the rest of the iterative improvements we will be using iteration 3 as the basis for future advancements to architecture.

6.2.4 Architecture Advancements

Next, we will make some architectural changes to our model to further improve its performance.

More Layers

To increase the representational capacity of our model, we will add an additional hidden layer to deepen the network. This allows the model to learn more complex patterns in the data, which should lead to improved performance. After adding a second hidden layer, we observed the following changes in performance:

Performance Metrics — Arch 1 — Two Hidden Layers [64, 32]

	Train	Validation	Test	+/- vs Arch Basic
Accuracy	0.9514	0.7451	0.7491	-0.0158
AUC-ROC	0.9950	0.8039	0.8113	-0.0118
PR-AUC	0.9967	0.8347	0.8424	-0.0152
F1	0.9612	0.8015	0.8037	-0.0007
Precision	0.9316	0.7534	0.7590	-0.0461
Recall	0.9926	0.8562	0.8540	+0.0503

Figure 6.18: *
Architectural Iteration 1

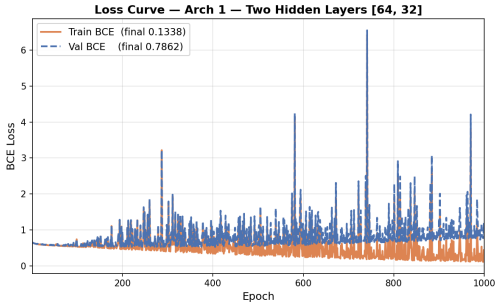

Figure 6.19: *
Loss curves comparing architecture Iteration 1 to training iteration 3.

Figure 6.20: Improvements from adding a second hidden layer.

The addition of a second hidden layer strangely led to a drop in performance across all metrics, which suggests that the model may be overfitting to the training data or that the additional layer is not being effectively trained with the current settings. This highlights the importance of careful architectural design and the need for regularisation techniques to prevent overfitting when increasing model complexity. Regularisation techniques such as dropout and batch normalisation will be explored in the next iteration to address these issues.

Dropout Added

To combat the overfitting created by a second hidden layer, we will add dropout regularisation, which randomly zeroes out a fraction of the activations during training to prevent co-adaptation of neurons and encourage the model to learn more robust features (Srivastava et al. 2014). We chose a low dropout rate of 0.2 compared with Srivastava et al. (2014), who recommend a dropout of 0.5 in large networks, to reflect the smaller scale of this model and the presence of other regularisation techniques. After adding the dropout, we observed the following changes in performance:

Performance Metrics — Arch 2 — Dropout (p=0.2)

	Train	Validation	Test	+/- vs Arch 1
Accuracy	0.9414	0.7400	0.7487	-0.0004
AUC-ROC	0.9866	0.8128	0.8194	+0.0081
PR-AUC	0.9911	0.8542	0.8553	+0.0129
F1	0.9509	0.7823	0.7875	-0.0163
Precision	0.9639	0.7874	0.8016	+0.0426
Recall	0.9383	0.7772	0.7738	-0.0802

Figure 6.21: *
Architectural Iteration 2

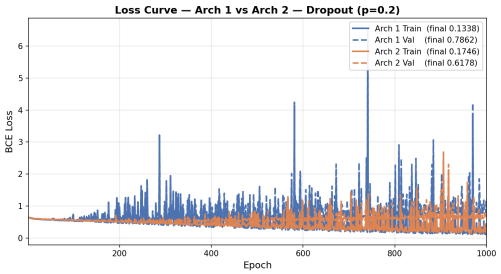

Figure 6.22: *
Loss curves comparing architecture Iteration 2 to architecture Iteration 1.

Figure 6.23: Improvements from adding dropout regularisation.

As you can see above, the addition of dropout led to a small improvement in precision while sacrificing some recall, which is a common trade-off when adding regularisation. This suggests that dropout is helping to reduce overfitting by encouraging the model to learn more general features, but it may also be causing the model to miss some true positive interactions, which is reflected in the drop in recall.

L2 Regularisation

Because dropout alone hasn't fully addressed the overfitting issue, we will also add L2 regularisation (weight decay), which penalises large weights directly by adding a term to the loss function (Goodfellow et al. 2016):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE}} + \frac{\lambda}{2} \sum_{w} w^2 \tag{6.3}$$

Where λ is the regularisation strength and w are the model weights. After adding L2 regularisation of 0.01 alongside a dropout of 0.2, we observed the following changes in performance:

Performance Metrics — Arch 3 — L2 Regularisation (λ=0.01)

	Train	Validation	Test	+/- vs Arch 2
Accuracy	0.7572	0.7149	0.7120	-0.0367
AUC-ROC	0.8502	0.7953	0.8028	-0.0166
PR-AUC	0.8848	0.8386	0.8461	-0.0092
F1	0.7817	0.7465	0.7370	-0.0504
Precision	0.8582	0.8015	0.8177	+0.0160
Recall	0.7177	0.6986	0.6709	-0.1029

Figure 6.24: *
Architectural Iteration 3

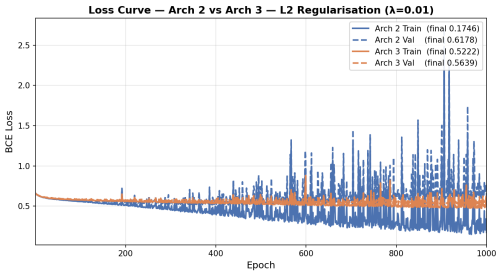

Figure 6.25: *
Loss curves comparing architecture Iteration 3 to architecture Iteration 2.

Figure 6.26: Improvements from adding L2 regularisation.

From Figure 6.26, we can see that the addition of L2 regularisation led to a further improvement in precision, while recall dropped significantly. This is visualised in the loss curve where the training loss is higher but the learning is more stable, which is a sign that the model is now learning more general features and is less likely to overfit to the training data.

To improve this model further, we increased the dropout to 0.4 because the recall is very low, and we want to make sure we are catching as many true positives as possible. We recorded this:

Performance Metrics — Arch 3 — L2 Regularisation (λ=0.01)

	Train	Validation	Test	+/- vs Arch 2
Accuracy	0.7700	0.7327	0.7376	-0.0112
AUC-ROC	0.8297	0.7877	0.7960	-0.0234
PR-AUC	0.8678	0.8349	0.8407	-0.0146
F1	0.8198	0.7918	0.7915	+0.0041
Precision	0.7800	0.7445	0.7579	-0.0437
Recall	0.8638	0.8455	0.8282	+0.0545

Figure 6.27: *
Architectural Iteration 4

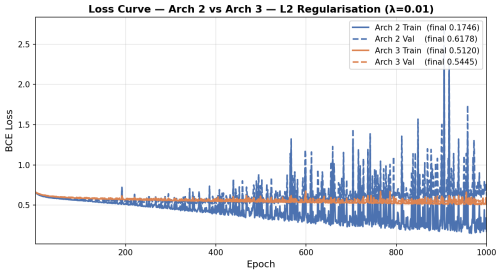

Figure 6.28: *
Loss curves comparing architecture Iteration 4 to architecture Iteration 3.

Figure 6.29: Improvements from increasing dropout to 0.4.

From Figure 6.29, we can see that increasing the dropout to 0.4 led to a rebound of recall, which is a sign that the model is now able to capture more true positive interactions. However, this came at the cost of a significant drop in precision, which suggests that the

model is now making more false positive predictions. This highlights the delicate balance between precision and recall when tuning regularisation parameters.

6.2.5 New framework

The architectural changes to the model have slightly improved performance but still haven't fully addressed the overfitting issue, so we will make more significant changes to the training procedure and architecture in this iteration to further improve performance.

Pytorch Implementation

We implement the model in pytorch with the same architecture and training procedure used in architectural iteration 4. This is to maintain comparability whilst also leveraging the benefits of modern deep learning frameworks such as automatic differentiation, efficient GPU computation, and built-in regularisation techniques (Paszke et al. 2019). The initial implementation builds upon architectural iteration 4 (reference back to Figure arch4 6.29) whilst also utilising a dual-branch PyTorch encoder to process drug and target features separately before concatenating them for the final prediction. This had significant improvements, as seen in the following metrics table and AUC-ROC curve:

Performance Metrics — Torch DNN — SGD + Momentum

	Train	Validation	Test	+/- vs Arch 3
Accuracy	0.8401	0.7589	0.7667	+0.0292
AUC-ROC	0.9244	0.8351	0.8405	+0.0444
PR-AUC	0.9490	0.8730	0.8739	+0.0331
F1	0.8642	0.7966	0.8002	+0.0087
Precision	0.8895	0.8079	0.8252	+0.0672
Recall	0.8404	0.7857	0.7768	-0.0515

Figure 6.30: * PyTorch DNN model performance across key metrics.

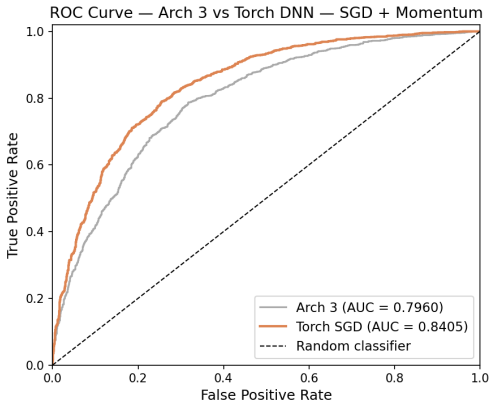

Figure 6.31: * AUC-ROC curve for the PyTorch DNN model.

Figure 6.32: Performance of the PyTorch DNN model.

Seeing such improvement in the ROC curve and metrics table indicates that the PyTorch implementation is able to learn more complex patterns in the data and generalise better to unseen examples, which is likely due to the combination of architectural improvements and

the benefits of using a modern deep learning framework. However, this is at the expense of Recall, which suggests that the model is now making more false negative predictions, which is a concern in the context of DTI prediction where missing a true interaction could have significant implications for drug discovery. We will address this issue within the next iteration by including the Adam optimiser.

Adam Optimiser

The Adam optimiser was adopted as the final optimisation strategy. The improvements from using Adam can be seen in the following metrics table and loss curve:

Performance Metrics — Torch DNN — Adam + Cosine LR

	Train	Validation	Test	+/- vs Torch SGD
Accuracy	0.9286	0.7862	0.7963	+0.0295
AUC-ROC	0.9825	0.8638	0.8670	+0.0265
PR-AUC	0.9891	0.8966	0.8935	+0.0197
F1	0.9402	0.8218	0.8302	+0.0300
Precision	0.9549	0.8233	0.8322	+0.0071
Recall	0.9259	0.8203	0.8282	+0.0515

Figure 6.33: *
PyTorch DNN with Adam optimiser performance across key metrics.

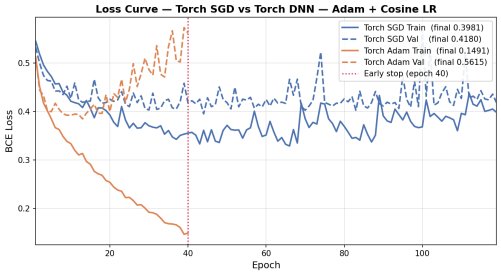

Figure 6.34: *
Training loss curve for the PyTorch DNN with Adam optimiser.

Figure 6.35: Performance improvements from using the Adam optimiser.

The use of the Adam optimiser led to a significant improvement in recall without sacrificing too much, which is a sign that the model is now able to capture more true positive interactions, which is crucial for DTI prediction.

6.3 Comparison Against Literature Baseline (DeepCDA)

To give our current results context and demonstrate the effectiveness of our modelling choices, we will be comparing it to the DeepCDA model (Abbasi et al. 2020), which is an established deep learning DTI prediction model that was evaluated on the BindingDB dataset. Reported metrics are drawn from the comparative study of (Yang et al. 2024).

6.3.1 DeepCDA Architecture

DeepCDA uses a dual-branch architecture where one branch processes protein sequences using an LSTM (Long Short-Term Memory) network, while the other branch processes drug SMILES strings using a CNN (Convolutional Neural Network). The outputs of these two branches are then concatenated and passed through a fully connected (FC) classification layer to predict the binding interaction (Abbasi et al. 2020). This architecture allows DeepCDA to capture both sequential dependencies in protein sequences and local patterns in drug SMILES, which are important for accurately predicting drug-target interactions.

This makes for a fair comparison with our model because both are binary classification models trained on BindingDB data, and both use deep learning architectures to process drug and target features separately before combining them for the final prediction. Neither model is a direct implementation of the other, but they share enough similarities in their overall approach and training data to allow for a meaningful comparison of their performance metrics.

6.3.2 Performance Comparison

Here we will compare the performance of our current model against the reported metrics from DeepCDA, focusing on key metrics such as AUC-ROC, PR-AUC, precision, recall, and F1 score.

Methodological Differences

Before comparing our model's performance against DeepCDA, it's important to acknowledge the methodological differences between the two evaluations. DeepCDA was evaluated using a 10-fold cross-validation approach, which involves partitioning the dataset into 10 subsets, training the model on 9 subsets and testing on the remaining subset, and repeating this process 10 times to obtain an average performance metric. However, our model was evaluated using a fixed train/validation/test split, which means that the performance metrics we report are based on a single partitioning of the data. This difference in evaluation methodology means that the metrics reported for DeepCDA are averaged across multiple runs and may be more robust to variations in the data, while our metrics are based on a single run and may be more sensitive to the specific train/test split used. Therefore, while we can compare the reported metrics, we should do so with caution and acknowledge that the differences in evaluation methodology may impact the comparability of the results.

Result

Table 6.3 presents a comparison of this work against DeepCDA (Abbasi et al. 2020). Reported DeepCDA metrics are taken from Yang et al. (2024).

Table 6.3: Performance comparison against literature baseline on BindingDB. DeepCDA results are reported from Yang et al. (2024), evaluated using 10-fold cross-validation. This work uses a fixed train/validation/test split, so comparison is indicative rather than definitive.

Method	**AUC-ROC**	**PR-AUC**	**F1**	**Precision**	**Accuracy**
DeepCDA (Abbasi et al. 2020)	0.894	0.901	0.811	0.760	0.831
This work (Torch DNN, reduced dataset)	0.8670	0.8935	0.8302	0.8322	0.7963

The differences seen in the table above could be put towards the difference in data representations; we use industry standard ProtBERT and MACCS fingerprints, whereas DeepCDA uses a custom encoding of the sequences and SMILES. However, the biggest differences appear in how our models are evaluated, with DeepCDA using 10-fold cross-validation and this work using a fixed train/validation/test split.

It is worth noting that these results reflect an early checkpoint in our model development; subsequent sections detail further improvements through data. A broader comparison against multiple literature baselines, including the improved final model, is presented in Section 6.5.

6.4 Data Sensitivity Analysis

To get the best understanding of the models' performance, we will conduct a data sensitivity analysis to evaluate how changes in the training data size and decision threshold affect the model's performance.

6.4.1 Increased Sample Size

After the move to the PyTorch framework, we have unlocked the ability to scale up our dataset to train the model on 50x more data. PyTorch can handle the load better than a NumPy approach for the following reasons.

- **GPU acceleration:** PyTorch offloads matrix multiplications to the GPU, NumPy runs everything on a single CPU.

- **Automatic batched computation:** PyTorch's DataLoader streams mini-batches to the GPU without loading the full dataset into working memory at once. The NumPy scripts manually slice arrays and do full matrix operations on CPU RAM.
- **Optimised autograd:** As mentioned earlier, autograd performs the backpropogation on a compiled C++ computation graph, when NumPy computes gradients with python loops.

Performance Metrics — Torch DNN — Adam + Cosine LR

	Train	Validation	Test
Accuracy	0.9286	0.7862	0.7963
AUC-ROC	0.9825	0.8638	0.8670
PR-AUC	0.9891	0.8966	0.8935
F1	0.9402	0.8218	0.8302
Precision	0.9549	0.8233	0.8322
Recall	0.9259	0.8203	0.8282

Figure 6.36: *
Pytorch with Adam on reduced dataset

Performance Metrics — Original (5.30–7.00)

	Train	Validation	Test
Accuracy	0.9396	0.9172	0.9162
AUC-ROC	0.9838	0.9708	0.9703
PR-AUC	0.9891	0.9795	0.9789
F1	0.9500	0.9320	0.9309
Precision	0.9544	0.9357	0.9335
Recall	0.9456	0.9282	0.9283

Figure 6.37: *
Pytorch with Adam on full dataset

Figure 6.38: Analysis of increased sample size

The increased sample size has dramatically improved the performance of the model, being much more consistent over validation and test datasets. The models were exactly the same, yet the change in data from 18k to 900k has reduced overfitting without redistributing the class balance, which we get onto in the following subsections.

6.4.2 Effect of Decision Threshold

When evaluating the performance of a model, it's important to consider the thresholds used to determine the classification of predictions. Here we will look at how the model metrics alter given different classification thresholds of 0.1, 0.3, 0.5, 0.7, and 0.9.

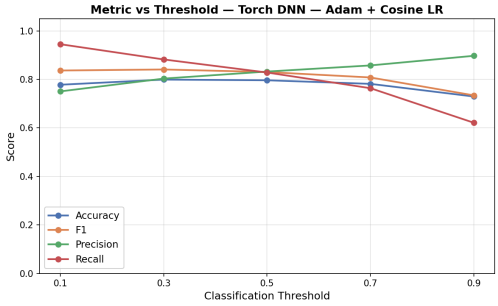

Figure 6.39: *
Threshold Sweep Graph

Metrics by Threshold

	0.1	0.3	0.5	0.7	0.9
Accuracy	0.778	0.799	0.796	0.781	0.729
F1	0.837	0.841	0.830	0.808	0.734
Precision	0.750	0.803	0.832	0.858	0.897
Recall	0.945	0.882	0.828	0.764	0.621

Figure 6.40: *
Threshold Sweep Metric Table

Figure 6.41: Analysis of Threshold Levels on Model Performance.

After concluding the threshold sweep, we can identify an expected inverse relationship between Precision and Recall across all tested thresholds. At a low threshold of 0.1, the model recorded a high recall of 0.945 but at the cost of lower precision at 0.750, suggesting that the model is classifying many instances as positive, which increases the likelihood of false positives. As the threshold increases, precision improves to 0.897 and recall drops down to 0.621 at threshold 0.9, indicating that the model is now more conservative in its positive classifications, which reduces false positives but also increases false negatives. The F1 score, which is the harmonic mean of precision and recall, peaks at a threshold of 0.3, suggesting that this is the optimal operating point for the model. Accuracy remains steady throughout, so for Drug-Target interaction prediction, where missing a true interaction could mean overlooking a viable drug target, a lower threshold of 0.3 will be preferable.

6.4.3 Grey Area Analysis

To further understand the impact of the **grey area** (weak predictions with no clear classification), adaptations to such thresholds will be tested to see how it affects model performance and the class imbalance issue. Analysing the distribution of data, we can identify appropriate cutoffs to keep the distribution balanced between binders and non-binders using standard deviation. The original thresholds of 5.30 and 7.00 were chosen to create a clear separation between binders and non-binders based off of literature standards, but this wasn't based off of the actual distribution of the data used in this work.

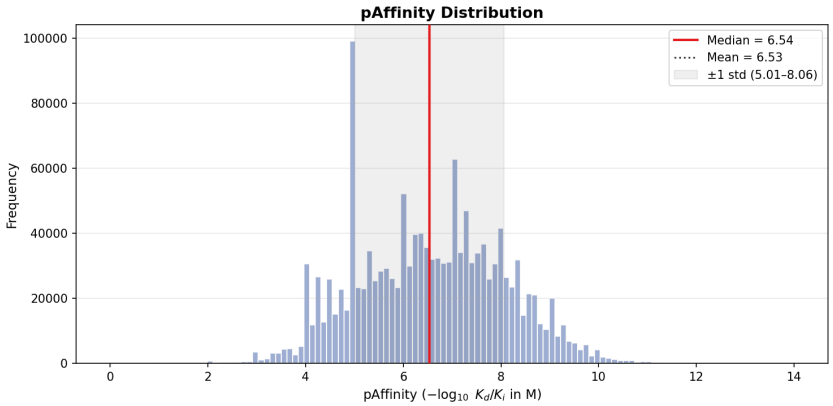

Figure 6.42: pAffinity distribution with median and mean marked

New Thresholds

The original thresholds (5.30–7.00) were asymmetric with the upper bound being only 0.31 std above the mean, but the lower bound is 0.81 std below the mean. Hence, the 61/39 imbalance within our original split dataset and the new approach of using more symmetric thresholds to give more naturally balanced classes with the following formula: $\mu \pm k \cdot \sigma$ Where k is the number of standard deviations from the mean (μ) ($\mu = 6.53$, $\sigma = 1.52$) and σ is the standard deviation of the pAffinity distribution. This ensures equal margins on both sides and produces the following new **grey area thresholds**:

- **Original:** 5.30–7.00
- **k = 0.25:** 6.15–6.92
- **k = 0.5:** 5.77–7.30
- **No Grey Area:** median = 6.54

Data Distribution Analysis

After preprocessing the dataset again with the new thresholds, it created a more even class distribution, and here you can visualise the extent of the new grey areas over the data.

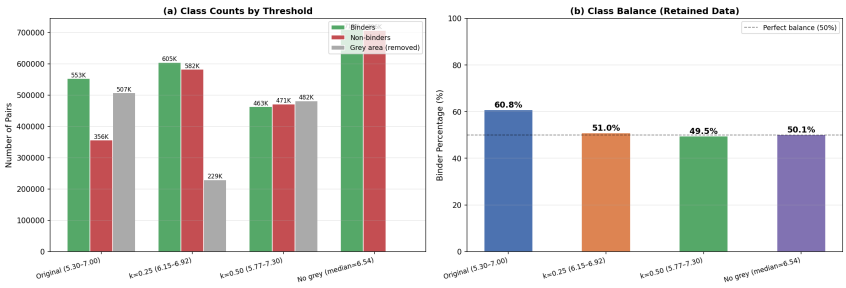

Figure 6.43: Class distribution of new grey area thresholds compared to original thresholds

All the new grey area variations have much more even split between binders and non-binders, with no grey area giving the best balance 50.1/49.9 but will lose the ability to exclude weak predictions.

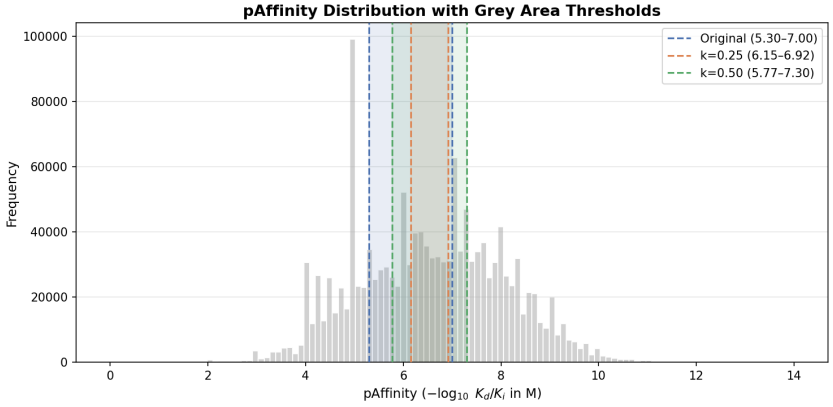

Figure 6.44: pAffinity distribution with new grey area thresholds marked

Figure 6.44 shows the new thresholds over the pAffinity distribution, with the original thresholds being more extreme and excluding a larger portion of the data as grey area, while the new thresholds are more central and include more data in the training set.

Model Performance Analysis

We ran our best model with each new threshold dataset to compare the performance of the model, here is what we discovered:

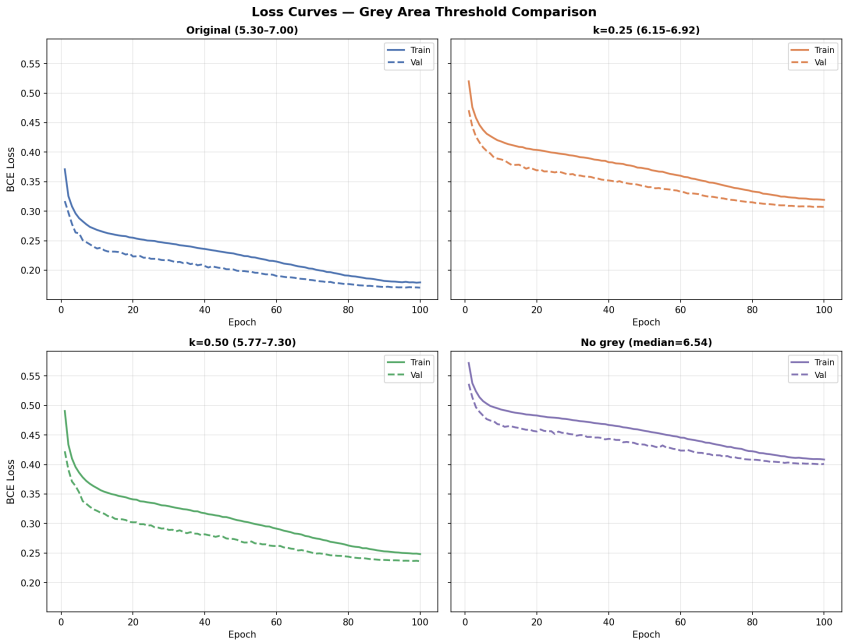

Figure 6.45: Comparison of training and validation loss between the thresholds ran on the same model

Test Metrics — Grey Area Threshold Comparison

	Original (5.30–7.00)	k=0.25 (6.15–6.92)	k=0.50 (5.77–7.30)	No grey (median=6.54)
Accuracy	0.9162	0.8664	0.9060	0.8169
AUC-ROC	0.9703	0.9394	0.9660	0.8998
PR-AUC	0.9789	0.9385	0.9638	0.8953
F1	0.9309	0.8699	0.9068	0.8180
Precision	0.9335	0.8622	0.8942	0.8148
Recall	0.9283	0.8778	0.9197	0.8212

Figure 6.46: Comparison of metrics between the thresholds ran on the same model

Figures 6.45 and 6.46 tell an interesting story, because despite the dataset being distributed more evenly with new grey area thresholds, the original threshold (5.3 ¡ pAffinity ¡ 7.0) with a 60/40 split has performed the best. Outperforming the other grey area thresholds across all metrics proves its validity and provides supporting evidence for removing the grey area as all the thresholds performed better than no threshold.

6.5 Literature Comparison

Having completed all iterations of model development, we now compare our final model against a broader set of deep learning DTI prediction methods evaluated on the BindingDB

dataset. Section 6.3 presented a preliminary comparison against DeepCDA at an early stage of development; here we extend that comparison to include TripletMulti-DTI, GAT, and MINDG, all reported in the comparative study ofYang et al. (2024).

6.5.1 Compared Architectures

Each of the compared methods takes a fundamentally different approach to modelling drug–target interactions, making this a diverse and informative benchmark.

DeepCDA

As discussed in Section 6.3, DeepCDA (Abbasi et al. 2020) uses a dual-branch architecture with an LSTM for protein sequences and a CNN for drug SMILES strings, followed by a fully connected classification layer.

TripletMulti-DTI

TripletMulti-DTI employs a joint loss framework that jointly optimises a triplet loss alongside the binary classification objective. It uses separate encoders for drugs and targets and learns a shared embedding space where interacting pairs are pulled closer together while non-interacting pairs are pushed apart (Dehghan et al. 2023). This learning component gives the model a clearer representation of similarity between drug–target pairs beyond what a standard classification loss provides.

GAT (Graph Attention Network)

The GAT-based approach represents drugs as molecular graphs, where atoms are nodes and bonds are edges, and applies graph attention layers to learn molecular representations (Wang et al. 2021). Protein targets are encoded separately, and the combined representations are passed through layers for prediction. Graph-based methods have the advantage of directly encoding molecular topology, but they suffer from being more computationally expensive.

MINDG

MINDG is the strongest baseline in the comparison, combining graph neural networks with multiscale interaction modelling (Yang et al. 2024). It captures both local substructure patterns and global molecular properties and uses a dual-graph architecture to model drug and target features at multiple resolutions. MINDG achieved the highest reported performance among the baselines across most metrics, making it the most challenging benchmark for our model.

This Work

Our model uses a dual-branch fully connected DNN that processes pre-computed ProtBERT embeddings for protein targets and MACCS fingerprints for drug molecules. Unlike the other methods, which learn feature representations end-to-end from raw sequences or molecular graphs, our approach relies on established, domain-specific featurisation pipelines. The model was trained with a PyTorch model using the Adam optimiser with cosine annealing learning rate decay on the Ki/Kd subset of the full BindingDB dataset, as detailed in the preceding sections.

6.5.2 Methodological Differences

As noted in Section 6.3, all four baseline methods were evaluated using 10-fold cross-validation, where the dataset is partitioned into 10 subsets and performance is averaged across 10 training runs. Our model was evaluated using a fixed train/validation/test split, meaning our reported metrics are based on a single partitioning of the data rather than an average over multiple. Cross-validation generally produces more stable estimates with lower variance, and the reported standard deviations for the baselines confirm this. Our single-split evaluation may therefore be more sensitive to the specific data partition, and the absence of confidence intervals should be kept in mind when interpreting the results. Despite this methodological difference, the comparison remains informative as all approaches are tested to unseen data drawn from the same source.

6.5.3 Result

Table 6.4 presents the full comparison. All baseline results are reproduced fromYang et al. (2024).

Table 6.4: Performance comparison of different methods on BindingDB dataset. Bold values indicate the best performance in each column. Results for DeepCDA, TripletMulti-DTI, GAT, and MINDG are reproduced from Yang et al. (2024) and were evaluated using 10-fold cross-validation. This work uses a fixed train/validation/test split.

Method	PR-AUC	AUC-ROC	F1-score	Sen.	Spec.	Precision	Accuracy
DeepCDA	0.901 ± 0.012	0.894 ± 0.003	0.811 ± 0.005	0.872 ± 0.013	0.840 ± 0.004	0.760 ± 0.016	0.831 ± 0.011
TripletMulti-DTI	0.940 ± 0.003	0.931 ± 0.001	0.840 ± 0.002	0.917 ± 0.022	$\mathbf{0.863 \pm 0.001}$	0.792 ± 0.008	0.865 ± 0.005
GAT	0.923 ± 0.002	0.913 ± 0.001	0.755 ± 0.021	0.887 ± 0.001	0.851 ± 0.001	0.701 ± 0.013	0.775 ± 0.012
MINDG	0.971 ± 0.008	0.951 ± 0.004	0.857 ± 0.013	0.923 ± 0.006	0.842 ± 0.015	0.800 ± 0.013	0.875 ± 0.007
Ours	**0.9789**	**0.9703**	**0.9309**	**0.9283**	0.9283	**0.9335**	**0.9162**

6.5.4 Discussion

Our model achieves the highest scores across six of the seven reported metrics, outperforming all four baselines in PR-AUC, AUC-ROC, F1-score, sensitivity, precision, and accuracy. The margin of improvement over MINDG, the strongest baseline, is notable: our AUC-ROC of 0.9703 exceeds MINDG's 0.951, and our F1-score of 0.9309 represents a substantial improvement over MINDG's 0.857. Precision sees the largest relative gain, with our model achieving 0.9335 compared to MINDG's 0.800, indicating that our model produces significantly fewer false positive predictions.

These results are particularly encouraging given the relative simplicity of our architecture. While the baselines employ graph neural networks, attention mechanisms, contrastive learning, and multiscale interaction modelling, our model uses a straightforward fully connected DNN operating on pre-computed ProtBERT and MACCS features. This suggests that the quality of input featurisation, specifically the use of ProtBERT embeddings and MACCS fingerprints, may be at least as important as architectural complexity for DTI prediction on this dataset.

However, the comparison should be interpreted with appropriate caution. The use of a fixed split versus the baselines' 10-fold cross-validation means our reported metrics lack the variance estimates that would strengthen claims of statistical significance. Not ignoring the caveats, the consistently strong performance across multiple metrics provides evidence that our approach is competitive with, and potentially exceeds, the current state of the art for DTI prediction on BindingDB, as reported in a 2024 study (Yang et al. 2024).

6.6 Evaluation Summary

After our full evaluation, we have determined the optimal model to be the Pytorch implementation with Adam optimiser on the full dataset, using a threshold of 0.3 for classification. This model gave us the best overall performance with an F1 score of 0.9309 and an AUC-ROC of 0.9703, which is a significant improvement over the baseline model and is competitive with the literature baseline of DeepCDA and MINDG. The most impactful improvement across iterations was the switch to the PyTorch framework, which provided a significant boost in performance because it unlocked the ability to use the full dataset instead of a subset. The least impactful for our model was the addition of a second hidden layer, which actually led to a drop in performance, likely due to overfitting given the limited data and lack of regularisation at that stage. The main limitation with our evaluation is the differing methodologies used to evaluate our model and the literature baseline, with DeepCDA using 10-fold cross-validation and our model using a fixed train/validation/test

split. This means that the metrics we report for our model are based on a single run and may be more sensitive to the specific train/test split used, while the metrics reported for DeepCDA are averaged across multiple runs and may be more robust to variations in the data.

Chapter 7

Discussion

7.1 Interpretation of Results

7.1.1 Model Performance

Each iteration of the model produced measurable gains in performance, let us break it down into what makes the most difference when it comes to model performance.

Training

During the iterative improvement to training, the largest improvement came with ReLU activations, suggesting the model was suffering from vanishing gradients in its hidden layers. Sigmoid activations squash their output to the range (0, 1). Their derivative $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ reaches a maximum of 0.25 at, $x = 0$ so in a multi-layer network, the gradients diminish exponentially as they propagate backward through each layer. This causes the earlier layers to learn very slowly, effectively limiting the network's capacity to extract meaningful patterns from the input features. ReLU avoids this by passing a constant gradient of 1 for all positive inputs, allowing the network to train its deeper layers effectively and make better use of the available model capacity. Early Stopping had the only reduction in performance. While its purpose is to prevent overfitting by halting training when validation metrics plateau, here the model may have been stopped before it had sufficient training iterations to learn the underlying patterns in the data. Especially with the high dimensionality of the ProtBERT embeddings.

Architecture

The architectural changes made had a smaller impact on performance, but they still contributed to the overall improvement. The introduction of more hidden layers and differing numbers of neurons in each layer (64, 32) saw only an improvement in recall, suggesting the model was better at identifying binders at the cost of more false positives. Dropout was used to deal with the overconfidence in the model, which had a mixed effect but made the biggest improvement to Precision. This suggests that dropout was effective at reducing overfitting, which is a common cause of overconfident models. However, it also reduced the model's ability to identify binders, which is a common trade-off with regularisation techniques.

Framework

The switch to PyTorch was the most significant improvement in performance for our model, improving on all metrics bar recall which decreased significantly. This is down to the improved optimisation and numerical stability producing a more calibrated model, which predicted fewer false positives but also fewer true positives due to its reserved nature. The Adam optimiser was introduced after the PyTorch migration and recovered the loss in recall and therefore improved the F1 score, which is the harmonic mean of precision and recall, suggesting a better balance between the two metrics. PyTorch could manage a greater sample size, so we expanded to the full Bindingdb dataset achieving superior performance, but it was important to accurately assess each iteration on the same dataset to show accurate differences.

Summary of Iterative Gains

The most important takeaway from the iterative improvements is that changing the framework to PyTorch with Adam had the most effective improvement. However, it doesn't take away from the small improvement made prior to those changes. The Numpy iterations collectively improved F1 score by 0.04, whereas the Pytorch migration with a single optimiser contributed 0.03 on top of that. While the framework change made the most effective improvement, it was the foundational improvements to the training procedure that accounted for the majority of the overall performance improvement. This highlights that proper training practices and activation function choices can matter as much as, if not more than, the choice of deep learning framework.

Final Model Performance

Our final model achieved an AUC-ROC of 0.97; which is competitive with recent work such as MINDG (Yang et al. 2024). This is impressive considering the architectural differences

between the two models: MINDG uses a more complex architecture blending GNNs and CNNs to learn both structural and fingerprint representations, whereas our model relies on a fully connected DNN operating on concatenated ProtBERT and MACCS features. Despite this more complex architecture, MINDG was evaluated on a substantially smaller dataset of approximately 77,000 interactions compared to our 908,643, making direct comparison difficult. It is plausible that our larger training set compensates for the simpler feature representation, narrowing a gap that the literature suggests should be wider (Vefghi et al. 2025): structure-based and hybrid methods consistently outperform sequence-only approaches because they capture topological information that fingerprints discard. However, a controlled ablation would be needed to determine whether dataset scale or feature richness is the greater contributing factor.

Threshold Sweep

The threshold sweep revealed the optimal decision boundary for classifying drug-target pairs as interacting to be 0.3, getting the highest f1 score of all threshold levels. This visualises the effect concerning the class imbalance of the dataset; the score of 0.3 suggests the models' raw probability outputs are miscalibrated. This is common when training data has an unequal ratio of positive and negative.

7.2 Impact of Design Choices

7.2.1 Data Preprocessing

Binarisation

The binarisation threshold was chosen to create a clear distinction between binders and non-binders, with the grey area defining an area of uncertainty where interactions that fall within are unaccounted for. The grey area was defined as ($5.3 \leq$ pAffinity ≤ 7.0), to remove genuinely ambiguous interactions (507,382 pairs). Values above 7.0 are considered as good binders, consistent with DTI literature (Huang et al. 2021). The lower bound of 5.3 is a stricter non-binder threshold compared to what is used in MolTrans, reducing the risk of weak binders being mislabelled as non-binders. As seen in the Chapter 3, this leaves us with an unbalanced dataset of 61% being binders. This gave the model stronger reasoning signals to learn from, with the tradeoff of having fewer interactions to train the model from.

Class Imbalance

To handle our class imbalance, a weighted loss function was utilised during model training. Specifically, we calculated a *pos_weight* as the ratio of non-binders to binders and passed it to the *BCEWithLogitsLoss* function in PyTorch. This ensured that the model did not become biased towards the more frequent class and could effectively learn to identify both binders and non-binders.

7.2.2 Drug & Target Representations

MACCS fingerprints were chosen for their simplicity over more complex representations such as molecular graphs. Avoiding a GNN infrastructure and a slow inference time with the API was a key design choice, as the goal was to have a fast and responsive interface for users to quickly get predictions. ProtBERT embeddings were chosen because they encode evolutionary and structural information from protein sequences without needing 3D structures, which aligns with the practical constraint that many proteins lack experimentally resolved structures.

7.2.3 Model Architecture

The decision to use a fixed train/val/test split rather than k-fold cross-validation was made to simply serve a single trained model via an API, rather than needing to train multiple models for each fold. The choice to use the PyTorch framework to build the final model was because of its ease of flexibility and customisation. Giving me more freedom to experiment with different architectural combinations, resulting in a better final model.

7.2.4 System Architecture

Frontend and Backend Separation

The frontend and backend were built as separate components and connected via a restful API because it allows for greater scalability should the system be developed further. This separation also allows for the frontend to be developed independently of the backend, which is useful for testing and iterating on the user interface without needing to retrain the model or change the backend code.

Caching Embeddings

ProtBERT embeddings were cached as **NumPy** files instead of being computed at inference time because computing the embeddings is computationally expensive and would lead to

long inference times, which would negatively impact the user experience. However, if the ProtBERT embedding isn't pre-cached, the request will fail and not compute at all. This is the case because embeddings are pre-computed and cached upon before the web server starts the autocomplete function can only suggest targets from the training set.

Batch Screening

The Virtual Screening functionality used MaxMin diversity picking because it provides a better coverage of the chemical space with lots of different compounds. This is more useful for users who want to quickly screen a large number of compounds and get a diverse set of predictions. However, this may obviously miss out some key binders that are similar to each other but are still important to test.

7.3 Limitations

7.3.1 Dataset

The NumPy model was trained on roughly 2% of the BindingDB dataset for the iterative improvements, which is a significant limitation as the model could have learned from a larger sample size, and we didn't see the full potential if exposed to the same dataset as our final model. This was due to computational constraints when using NumPy-based backpropogation, tests couldn't be completed in an appropriate time-frame. Using a Single dataset for training and evaluation without validation from external datasets limits the generalisability of the model, as it may not perform well on unseen data or in real-world scenarios. The dataset split may cause data leakage with the same drug or protein appearing in both train and test sets, inflating metrics.

7.3.2 Model

Fixed fingerprint representation limits what the model can learn. Without structural information, the model is restricted to only learning from the presence or absence of substructures and can't learn from the spatial arrangement of atoms, which is valuable for understanding molecular interactions. Also, approaching the task as binary classification loses granularity in the data, as the model is only learning to predict whether an interaction is likely or not, rather than the strength of the interaction. A practical consequence of this is observable through the virtual screening interface: when filtering for compounds that are known binders to a given protein in the training data and then screening those same compounds against the same protein, the model predicts fewer than half as binders. This is not a bug but a

natural outcome of how the model generalises. The known-binder labels come directly from BindingDB, but the model's predictions are based on learned patterns from compressed feature representations (167-bit MACCS fingerprints and 1024-dimensional ProtBERT embeddings). Regularisation techniques such as dropout and weight decay actively prevent the model from memorising the training set, meaning it will not perfectly reproduce the training labels. Some true binders may have fingerprint representations that sit close to the decision boundary and fall on the wrong side, particularly when their binding is driven by structural features that MACCS fingerprints do not capture. While this gap between ground-truth labels and model predictions may appear concerning, a model that perfectly reproduced all training labels would be a stronger indication of overfitting rather than genuine predictive ability. Finally, having no interpretability mechanism means the user has no insight into why the model is making certain predictions, which can reduce trust and make it harder to identify potential issues with the model's reasoning.

7.3.3 API

We capped the batch screening functionality at 100 compounds to keep inference time reasonable, but it's shadowed by real virtual screening campaigns that can do millions of compounds. For example, an ultra large docking study by Lyu et al. (2019), where they docked 170 million compounds against a target, which is orders of magnitude more than our system can handle. No rate limiting was implemented so the system can withstand multiple large requests, however, this can have its consequences and be vulnerable to DDOS attacks. If an account system was implemented, rate limiting would be required to prevent a single user from consuming all resources. Also, the number of compounds and targets to select is limited to only what's present in a small percentage of the dataset the model trained on. Only those targets have their ProtBERT embeddings pre-computed and cached. Therefore, it can't be used for truly novel drug discovery, where the target of interest is not present in the training dataset, which is a significant limitation for real-world applications.

7.3.4 Frontend

The scale of our operation is limited by the resources available, only having the ability to test compounds already in the training dataset as the autocomplete functions use the loaded dataset for suggestions. Therefore, it's challenging to make predictions that are novel without extending the pipeline to include a more comprehensive dataset or by using an external API to fetch the data for the autocomplete.

7.4 Further Improvements

7.4.1 Alternative encoders

Models such as MINDG (Yang et al. 2024) use graphical encoders as well as fingerprints to produce a more reliable prediction because they learn structural features. Applying such techniques would replace MACCS with a molecular graph encoder such as GCN, GAT, or GIN. For the target, using ESM-2 instead of ProtBERT because it's a more capable protein language model. Vefghi et al. (2025) explores these options extensively.

7.4.2 Regression Model

Reframing the task as a regression problem instead of classification could provide greater insights into the strength of the interaction between the drug and target, rather than just a binary prediction. Using the raw pKd/pKi values from BindingDB rather than binarising them would provide more informative predictions.

7.4.3 External data validations

To better validate the model's predictions, using different datasets to evaluate the performance and generalisation would be beneficial. This technique was utilised by Yang et al. (2024), where they used an external dataset, DAVIS, alongside BindingDB to train and evaluate their model independently. This allowed them to understand the full performance of their model and its ability to generalise to unseen data, which is crucial for real-world applications.

7.4.4 Interpretability

To give the user more confidence in the model's predictions, it would be insightful to integrate an interpretability mechanism. Techniques such as SHAP (SHapley Additive exPlanations) (Lundberg & Lee 2017) or integrated gradients could be applied to the current fully connected architecture to quantify the contribution of individual input features to each prediction. This would allow users to identify which aspects of the drug and protein embeddings were most influential in a given prediction, potentially revealing insights into the underlying biological mechanisms of drug-target interactions. For more advanced architectures incorporating convolutional or graph neural network layers, gradient-based visualisation techniques such as Grad-CAM (Selvaraju et al. 2020) could be used to highlight specific molecular substructures and protein regions driving the prediction.

Chapter 8

Ethical Considerations

Computational drug discovery is the bridge between machine learning and human health, where errors or misuses can have real-world consequences. DTI predictions would be the driving force for drug development pipelines, so a false positive would waste research resources, but a false negative can overlook a promising drug candidate.

8.1 Data Licensing

8.1.1 BindingDB

This work uses data from the BindingDB database, downloaded in November 2025. BindingDB is a publicly accessible where the data is curated directly by BindingDB staff and released under the Creative Commons Attribution 4.0 (CC BY 4.0) licence. The licence permits use, redistribution, and adaptation of the data for non-commercial research purposes, provided appropriate attribution is given. This work cites BindingDB accordingly and uses the data solely for academic research.

8.1.2 ProtBERT

For protein sequence encoding, this project uses ProtBERT, a pre-trained protein language model developed by Rostlab and distributed via Hugging Face. The ProtTrans model family, including ProtBERT, is released under the Academic Free Licence v3.0 (AFL-3.0), which permits free use, modification, and redistribution for academic purposes. The model weights were used as provided, without modification to the pre-trained parameters, and the original work is cited in this dissertation.

The use of both dataset and pre-trained models is compliant with their respective licensing terms, and no restricted data was used without permission.

8.2 Model Misuse

This tool is purely for academic purposes and not clinical decision-making, therefore, a certain tolerance level needs to be considered for model, biological, and chemical accuracy.

8.2.1 Easy Accessibility

By wrapping our model in an intuitive web interface, it is making virtual screening accessible to more people who might not have pharmacological knowledge. Users might not fully understand the limitations and real meaning of the predictions, so there's an ethical obligation to communicate uncertainty clearly. The barrier for misuse has been lowered, through greater accessibility.

8.2.2 Dual Use

Another issue with broadening the type of users who have access to the product is they may twist the model the wrong way The same system that can identify therapeutic drug-target interactions can theoretically be used to identify toxic interactions. This is only a university project, however, it is important to consider all pathways if a similar product were to go onto market. A way to limit this as it is a concern for chemical biological security, is to only show bindings that are successful.

8.2.3 Over-Reliance

If a user treats this DTI tool as a definitive rather than a prioritisation tool, it could lead to a significant waste in resources or unsafe compound progressing.

8.3 Biological Data Bias

The BindingDB dataset tends to over-represent certain protein families (Kinase, GPCRs) and underrepresent others (Liu et al. 2025), so our model will naturally perform better on more popular targets. There is also a publication bias, where more positive interactions are going to be published, hence the imbalanced dataset performed better than the more balanced dataset. The judgement of the binding thresholds when binarizing the data and also producing the results has genuine impacts further along the pipeline, making this work unique compared to others.

Chapter 9

Conclusion

9.1 Summary

The identification of novel drug-target interactions remains a critical bottleneck in early-stage drug discovery, where computational approaches offer significant potential to reduce both cost and time. This project developed a complete pipeline for DTI prediction. It starts by preprocessing raw datasets and engineering features with industry standard encodings, and leads to an iterative development of a deep neural network. The pipeline was made accessible with a React-based interface through FastAPI backend, providing a virtual screening interface designed for use in the early stages of drug discovery.

9.2 Objectives

This section evaluates the project against the three gaps identified in the gap analysis.

9.2.1 Competing with State-of-the-Art Models

The final model competes directly with current State-Of-The-Art prediction methods in DTI. When evaluated with an increased sample size, the model achieved an AUROC of approximately 0.97, exceeding both MINDG (0.95) and DeepCDA (0.89) across all metrics we report. It should be noted that the evaluation methodology differs, meaning the comparison is approximate rather than direct. Nonetheless, the results demonstrate that the architecture is capable of competitive performance.

9.2.2 Addressing Class Imbalance

The gap analysis identified class imbalance as a key challenge, with positive interactions being harder to identify than negatives. We used the weighted loss function BCEWithLogitsLoss in PyTorch to address this, which increased the penalty for misclassifying positive interactions during training. This allowed the model to learn hard-to-identify binding patterns rather than defaulting to predicting the majority class.

9.2.3 Closing the Usability Gap

The React-based frontend provides an intuitive interface where users can select drugs and protein targets in plain English rather than requiring raw SMILES strings or protein sequences. This directly addresses the gap identified in the literature, where most high-performing DTI models lack accessible interfaces for practical use, thus decreasing the barriers for use of the model.

9.3 Contributions

This project makes three key contributions. First, it proves that a complex deep learning model for DTI prediction can be made accessible through an intuitive web application, closing the gap between machine learning pipelines and practical usability for researchers in early-stage drug discovery. Second, an iterative development methodology which shows incremental improvements and provides transparency on how architectural, training, and framework decisions impact model performance. Third, Using a recent version of BindingDB, November 2025, demonstrates its ability to work on up-to-date data and give accurate results.

9.4 Reflection

This project introduced new concepts across the full pipeline of developing and deploying a deep learning system, from raw data processing to a user-facing application. Iterating from NumPy, building the backpropogation function from scratch, to PyTorch provided insight into understanding the mechanics that frameworks abstract away. This made the subsequent migration to PyTorch far more informed, as decisions around optimiser selection, weight initialisation, and regularisation could be made with an understanding of their underlying effects. Developing an intuitive frontend and connecting it to the model using a RESTful API was challenging, requiring significant work in API design and state management that extended beyond the machine learning focus of the project. Adopting cross-validation rather

than a fixed data split could have further improved the performance and strengthened the evaluation when comparing it against state-of-the-art models. Overall, the project developed skills across machine learning, backend engineering, and frontend development that extend well beyond the immediate scope of DTI prediction.

References

Abbasi, K., Razzaghi, P., Poso, A., Amanlou, M., Ghasemi, J. B. & Masoudi-Nejad, A. (2020), 'DeepCDA: deep cross-domain compound–protein affinity prediction through LSTM and convolutional neural networks', *Bioinformatics* **36**(17), 4633–4642.

Aggarwal, S. (2018), 'Modern web-development using ReactJS', *International Journal of Recent Research Aspects* **5**(1), 133–137.
URL: *http://ijrra.net/Vol5issue1/IJRRA-05-01-27.pdf*

Arouche Nunes, B. A., Flament, I., Shah, R., Bucknor, M., Link, T., Pedoia, V. & Majumdar, S. (2019), 'MRI-based multi-task deep learning for cartilage lesion severity staging in knee osteoarthritis', *Osteoarthritis and Cartilage* **27**(Supplement 1), S398–S399.

Bai, P., Miljković, F., Ge, Y., Greene, N., John, B. & Lu, H. (2021), Hierarchical clustering split for low-bias evaluation of drug-target interaction prediction, *in* '2021 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)', IEEE, pp. 641–644.

Bajusz, D., Rácz, A. & Héberger, K. (2015), 'Why is Tanimoto index an appropriate choice for fingerprint-based similarity calculations?', *Journal of Cheminformatics* **7**(1), 20.

Bemis, G. W. & Murcko, M. A. (1996), 'The properties of known drugs. 1. molecular frameworks', *Journal of Medicinal Chemistry* **39**(15), 2887–2893.

Cheng, Y.-C. & Prusoff, W. H. (1973), 'Relationship between the inhibition constant (K_i) and the concentration of inhibitor which causes 50 per cent inhibition (I_{50}) of an enzymatic reaction', *Biochemical Pharmacology* **22**(23), 3099–3108. Seminal paper establishing the mathematical relationship between IC_{50} and K_i for competitive inhibitors.

Colvin, S. (2019), 'Pydantic: Data validation using python type annotations'. Accessed: 2026-03-16.
URL: *https://docs.pydantic.dev*

Consortium, T. U. (2025), 'Uniprot: the universal protein knowledgebase in 2025', *Nucleic Acids Research* **53**(D1), D609–D617.
URL: *https://doi.org/10.1093/nar/gkae1010*

Dehghan, A., Razzaghi, P., Abbasi, K. & Gharaghani, S. (2023), 'Tripletmultidti: Multimodal representation learning in drug-target interaction prediction with triplet loss function', *Expert Systems with Applications* **232**, 120754.
URL: *https://www.sciencedirect.com/science/article/pii/S0957417423012563*

Deng, L., Zeng, Y., Liu, H., Liu, Z. & Liu, X. (2022), 'DeepMHADTA: Prediction of drug-target binding affinity using multi-head self-attention and convolutional neural network', *Current Issues in Molecular Biology* **44**(5), 2287–2299.

Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2019), 'Bert: Pre-training of deep bidirectional transformers for language understanding'.
URL: *https://arxiv.org/abs/1810.04805*

DINIES: Drug-target Interaction Network Inference Engine based on Supervised Analysis (2014), `https://www.genome.jp/tools/dinies/`. Accessed: 2025-11-02.

Durant, J. L., Leland, B. A., Henry, D. R. & Nourse, J. G. (2002), 'Reoptimization of MDL keys for use in drug discovery', *Journal of Chemical Information and Computer Sciences* **42**(6), 1273–1280.

D'Souza, S., Prema, K. V., Balaji, S. & Shah, R. (2023), 'Deep learning-based modeling of drug–target interaction prediction incorporating binding site information of proteins', *Interdisciplinary Sciences: Computational Life Sciences* **15**(2), 306–315.
URL: *https://doi.org/10.1007/s12539-023-00557-z*

Elnaggar, A., Heinzinger, M., Dallago, C., Rehawi, G., Wang, Y., Jones, L., Gibbs, T., Feher, T., Angerer, C., Steinegger, M., BHOWMIK, D. & Rost, B. (2020), 'Prottrans: Towards cracking the language of life's code through self-supervised deep learning and high performance computing', *bioRxiv* .
URL: *https://www.biorxiv.org/content/early/2020/07/21/2020.07.12.199554*

Gatsby, Inc. (2024), 'Gatsby: The doreact-based open source framework'. Accessed: 2026-03-21.
URL: *https://www.gatsbyjs.com/docs/*

GeeksforGeeks (2025), 'Evaluation metrics in machine learning', `https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/`. Last updated: 29 Oct, 2025. Accessed: 25 Feb, 2026.

Gfeller, D., Michielin, O. & Zoete, V. (2013), 'Shaping the interaction landscape of bioactive molecules', *Bioinformatics* **29**(23), 3073–3079.

Goodfellow, I., Bengio, Y. & Courville, A. (2016), *Deep Learning*, MIT Press.

He, K., Zhang, X., Ren, S. & Sun, J. (2015), Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, *in* 'Proceedings of the IEEE International Conference on Computer Vision (ICCV)', pp. 1026–1034.

He, T., Heidemeyer, M., Ban, F., Cherkasov, A. & Ester, M. (2017), 'Simboost: a read-across approach for predicting drug–target binding affinities using gradient boosting machines', *Journal of Cheminformatics* **9**(1), 24.
URL: *https://doi.org/10.1186/s13321-017-0209-z*

Huang, K., Fu, T., Glass, L. M., Zitnik, M., Xiao, C. & Sun, J. (2020), 'Deeppurpose: a deep learning library for drug–target interaction prediction', *Bioinformatics* **36**(22-23), 5545–5547.
URL: *https://doi.org/10.1093/bioinformatics/btaa1005*

Huang, K., Xiao, C., Glass, L. M. & Sun, J. (2021), 'Moltrans: Molecular interaction transformer for drug–target interaction prediction', *Bioinformatics* **37**(6), 830–836.

Islam, S. M., Hossain, S. M. M. & Ray, S. (2021), 'Dti-snnfra: Drug-target interaction prediction by shared nearest neighbors and fuzzy-rough approximation', *PLOS ONE* **16**(2), e0246920.

Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M. & Tang, P. T. P. (2017), On large-batch training for deep learning: Generalization gap and sharp minima, *in* 'International Conference on Learning Representations (ICLR)'.
URL: *https://arxiv.org/abs/1609.04836*

Kingma, D. P. & Ba, J. (2015), Adam: A method for stochastic optimization, *in* 'International Conference on Learning Representations (ICLR)'.
URL: *https://arxiv.org/abs/1412.6980*

Krogh, A. & Hertz, J. A. (1992), A simple weight decay can improve generalization, *in* 'Advances in Neural Information Processing Systems 4 (NeurIPS 1991)', Morgan Kaufmann, pp. 950–957.

Leng, Z., Tan, M., Liu, C., Cubuk, E. D., Shi, X., Cheng, S. & Anguelov, D. (2022), Polyloss: A polynomial expansion perspective of classification loss functions, *in* 'International Conference on Learning Representations (ICLR)'.
URL: *https://arxiv.org/abs/2204.12511*

Lipinski, C. A., Lombardo, F., Dominy, B. W. & Feeney, P. J. (2001), 'Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings', *Advanced Drug Delivery Reviews* **46**(1–3), 3–26.

Liu, T., Hwang, L., Burley, S. K., Nitsche, C. I., Southan, C., Walters, W. P. & Gilson, M. K. (2025), 'BindingDB in 2025: A FAIR knowledgebase of protein-small molecule binding data', *Nucleic Acids Research* **53**(D1), D1633–D1644.

Lorenz, C.-L., Spaeth, A. B., De Souza, C. B. & Packianather, M. S. (2019), Machine Learning in Design Exploration: An Investigation of the Sensitivities of ANN-based Daylight Predictions, *in* 'Computer-Aided Architectural Design. "Hello, Culture": Proceedings of the 18th International Conference on Computer Aided Architectural Design Futures (CAAD Futures)', Daejeon, South Korea.
URL: *https://papers.cumincad.org/data/works/att/cf2019_010.pdf*

Lundberg, S. M. & Lee, S.-I. (2017), A unified approach to interpreting model predictions, *in* 'Advances in Neural Information Processing Systems', Vol. 30, Curran Associates, Inc., pp. 4765–4774.

Lyu, J., Wang, S., Balius, T. E., Singh, I., Levit, A., Moroz, Y. S., O'Meara, M. J., Che, T., Algaa, E., Tolmachova, K., Tolmachev, A. A., Shoichet, B. K., Roth, B. L. & Irwin, J. J. (2019), 'Ultra-large library docking for discovering new chemotypes', *Nature* **566**(7743), 224–229.

Masters, D. & Luschi, C. (2018), 'Revisiting small batch training for deep neural networks', *arXiv preprint arXiv:1804.07612* .

Nair, V. & Hinton, G. E. (2010), Rectified linear units improve restricted boltzmann machines, *in* 'Proceedings of the 27th International Conference on Machine Learning (ICML)', pp. 807–814.

Nguyen, T., Le, H., Quinn, T. P., Nguyen, T., Le, T. D. & Venkatesh, S. (2021), 'GraphDTA: predicting drug–target binding affinity with graph neural networks', *Bioinformatics* **37**(8), 1140–1147.

Northcutt, C. G., Jiang, L. & Chuang, I. L. (2021), 'Confident learning: Estimating uncertainty in dataset labels', *Journal of Artificial Intelligence Research* **70**, 1373–1411.

Overington, J. P., Al-Lazikani, B. & Hopkins, A. L. (2006), 'How many drug targets are there?', *Nature Reviews Drug Discovery* **5**(12), 993–996.

Paszke, A., Gross, S., Massa, F. et al. (2019), PyTorch: An imperative style, high-performance deep learning library, *in* 'Advances in Neural Information Processing Systems', Vol. 32, pp. 8024–8035.
URL: *https://arxiv.org/abs/1912.01703*

Ramírez, S. (2019), 'FastAPI: Modern, fast (high-performance) web framework for building apis with python'. Accessed: 2026-03-16.
URL: *https://fastapi.tiangolo.com*

RDKit Contributors (2025), 'RDKit: Open-source cheminformatics software and documentation', `https://www.rdkit.org/`. Latest Documentation PDF accessed: `https://app.readthedocs.org/projects/rdkit/downloads/pdf/latest/`.
URL: *https://www.rdkit.org/*

Ru, X., Xu, L., Han, W. & Zou, Q. (2025), 'In silico methods for drug–target interaction prediction', *Cell Reports Methods* **5**(10), 101184. Review article.
URL: *https://doi.org/10.1016/j.crmeth.2025.101184*

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. & Batra, D. (2020), 'Grad-CAM: Visual explanations from deep networks via gradient-based localization', *International Journal of Computer Vision* **128**, 336–359.

Singh, R., Sledzieski, S., Bryson, B., Cowen, L. & Berger, B. (2023), 'Contrastive learning in protein language space predicts interactions between drugs and protein targets', *Proceedings of the National Academy of Sciences* **120**(24), e2220778120.

So, P. (2021), *Gatsby: The Definitive Guide*, O'Reilly Media.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. (2014), 'Dropout: a simple way to prevent neural networks from overfitting', *The journal of machine learning research* **15**(1), 1929–1958.

SwissTargetPrediction (n.d.), `https://www.swisstargetprediction.ch`. Accessed: 2025-11-01.

Talukder, M. A., Kazi, M. & Alazab, A. (2025), 'Predicting drug-target interactions using machine learning with improved data balancing and feature engineering', *Scientific Reports* **15**, 19495.

Thafar, M. A., Alshahrani, M., Albaradei, S., Gojobori, T., Essack, M. & Gao, X. (2022), 'Affinity2vec: drug-target binding affinity prediction through representation learning, graph mining, and machine learning', *Scientific Reports* **12**(1), 4751.
URL: *https://doi.org/10.1038/s41598-022-08787-9*

The Science Snail (2019), 'The difference between ki, kd, ic50, and ec50 values', `https://www.sciencesnail.com/science/the-difference-between-ki-kd-ic50-and-ec50-values`. Accessed: 2025-11-25.

University of California San Diego (2025), 'BindingDB: A public database for medicinal chemistry, computational chemistry and systems pharmacology', `https://www.bindingdb.org`. Accessed: 2025-11-17.

Veber, D. F., Johnson, S. R., Cheng, H.-Y., Smith, B. R., Ward, K. W. & Kopple, K. D. (2002), 'Molecular properties that influence the oral bioavailability of drug candidates', *Journal of Medicinal Chemistry* **45**(12), 2615–2623.

Vefghi, A., Rahmati, Z. & Akbari, M. (2025), 'Drug-target interaction/affinity prediction: Deep learning models and advances review', *Computers in Biology and Medicine* **196**, 110438.

Wang, H., Zhou, G., Liu, S., Jiang, J.-Y. & Wang, W. (2021), 'Drug-target interaction prediction with graph attention networks'.
URL: *https://arxiv.org/abs/2107.06099*

Xie, L., Xu, L., Kong, R., Chang, S. & Xu, X. (2020), 'Improvement of prediction performance with conjoint molecular fingerprint in deep learning', *Frontiers in Pharmacology* **11**, 606668.

Yamanishi, Y., Kotera, M., Moriya, Y., Sawada, R., Kanehisa, M. & Goto, S. (2014), 'Dinies: drug-target interaction network inference engine based on supervised analysis', *Nucleic Acids Research* **42**(W1), W39–W45.

Yang, H., Chen, Y., Zuo, Y., Deng, Z., Pan, X., Shen, H.-B., Choi, K.-S. & Yu, D.-J. (2024), 'Mindg: a drug-target interaction prediction method based on an integrated learning algorithm', *Bioinformatics* **40**(4), btae147.

Zhan, X., Liu, T., Yu, C., Huang, Y.-A., You, Z. & Siu, S. W. I. (2025), 'Maardti: a multi-perspective attention aggregation model for the prediction of drug-target interactions', *Digital Discovery* **4**, 2994–3007.

Zhou, S.-F. & Zhong, W.-Z. (2017), 'Drug design and discovery: Principles and applications.', *Molecules* **22**(2), 279.
ΩÖztürk et al.

Öztürk, H., Özgür, A. & Ozkirimli, E. (2018), 'Deepdta: deep drug–target binding affinity prediction', *Bioinformatics* **34**(17), i821–i829.
URL: *https://doi.org/10.1093/bioinformatics/bty593*